Python Activity-Based Learning

1.

Attendance Recovery Computation: Calculate current attendance percentages and determine the minimum consecutive classes required to reach an eligibility threshold under different future attendance scenarios.

Sol:

Problem statement

Calculate a student's current attendance percentage and determine the minimum number of consecutive classes the student must attend to reach a required attendance percentage.

Objectives

- Calculate attendance percentage.
- Use a loop to find the minimum future classes needed.
- Display a clear eligibility result.

Assumptions

- Classes held and classes attended are non-negative integers.
- Classes attended cannot exceed classes held.
- The required percentage is between 0 and 100.

Algorithm

1. Read classes held, classes attended, and required percentage.
2. Validate the inputs.
3. Calculate current attendance percentage.
4. If the target is already reached, print 0 additional classes.
5. Otherwise, repeatedly add one attended class and one held class until the target is reached.
6. Print the percentage and additional classes needed.

Pseudocode

1. START

2. Read Input held, attended, target

3. if held < 0 OR attended < 0 OR attended > held OR target < 0 OR target > 100:
    print Invalid input

4. else:
    if held == 0:
        current = 0
    else:
        current = attended / held * 100

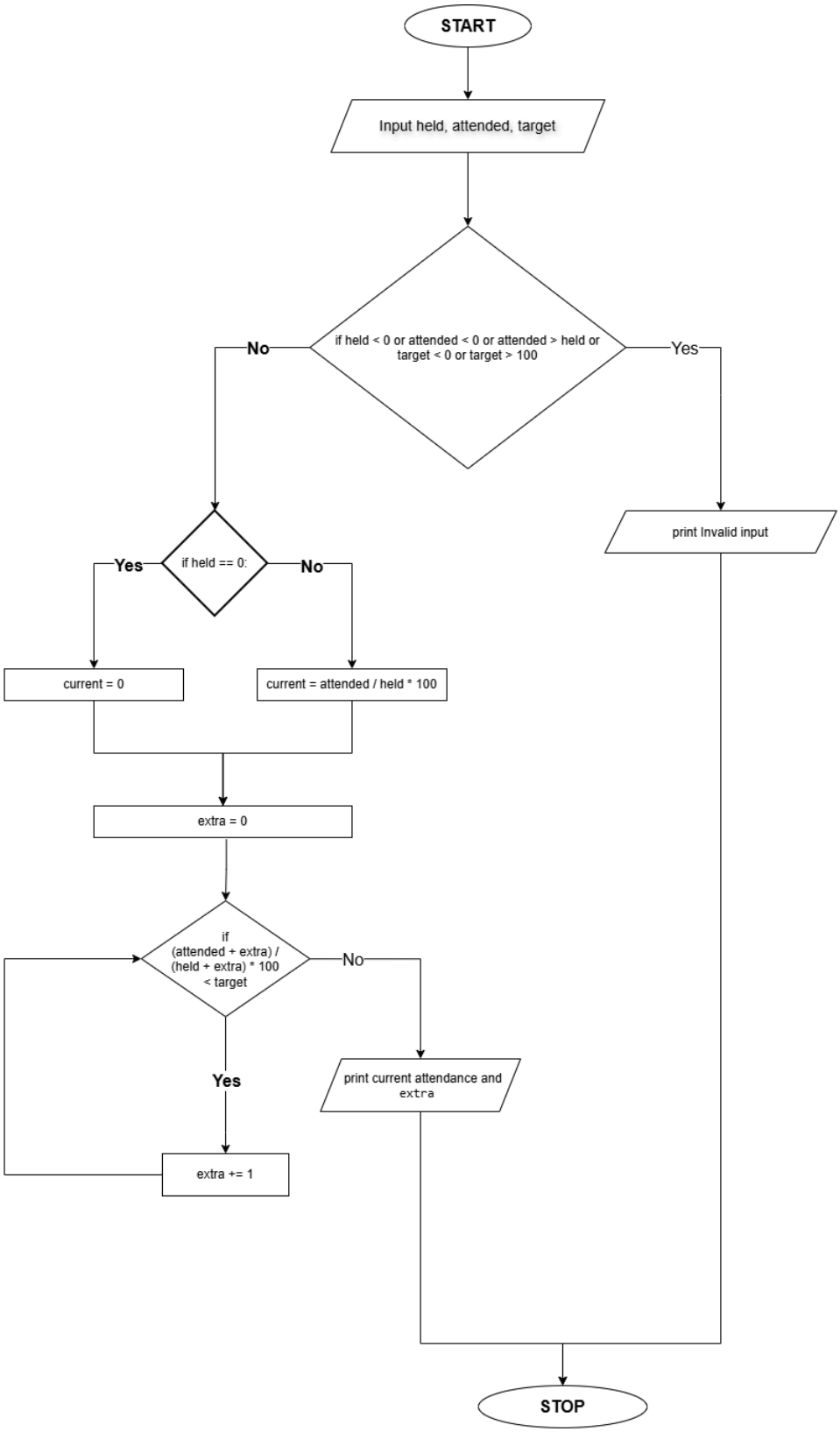
5. Set extra = 0

6. Now, while (attended + extra) / (held + extra) * 100 < target:
    extra+=1

7. print percentage, extra

8. STOP

Flowchart


START
Input held, attended, target
if held < 0 or attended < 0 or attended > held or target < 0 or target > 100
No
Yes
print Invalid input
if held == 0:
Yes
No
current = 0
current = attended / held * 100
extra = 0
if (attended + extra) / (held + extra) * 100 < target
No
Yes
print current attendance and extra
extra += 1
STOP

Source Code

```
held = int(input("Enter total classes held: "))
attended = int(input("Enter classes attended: "))
target = float(input("Enter required attendance percentage: "))

if held < 0 or attended < 0 or attended > held or target < 0 or target > 100:
    print("Invalid input.")
else:
    if held == 0:
        current = 0
    else:
        current = attended / held * 100
extra = 0
while (attended + extra) / (held + extra) * 100 < target:
    extra += 1
print(f"Current attendance: {current:.2f}%")
print("Additional classes to attend:", extra)
```

Test Cases

- Input: 40; 32; 75
  - Expected result: Current attendance: 80.0 %; Additional classes to attend: 0
- Input: 20; 12; 75
  - Expected result: Current attendance: 60.0 %; Additional classes to attend: 12
- Input: 10; 10; 75
  - Expected result: Current attendance: 100.0 %; Additional classes to attend: 0

Output

```python
held = int(input("Enter total classes held: "))
attended = int(input("Enter classes attended: "))
target = float(input("Enter required attendance percentage: "))

if held < 0 or attended < 0 or attended > held or target < 0 or target > 100:
    print("Invalid input.")
else:
     if held == 0:
        current = 0
     else:
        current = attended / held * 100
extra = 0
while (attended + extra) / (held + extra) * 100 < target:
    extra += 1
print(f"Current attendance: {current:.2f}%")
print("Additional classes to attend:", extra)
```

```
un  1 ×

D:\Python\.venv\Scripts\python.exe D:\Python\ABL\1.py
Enter total classes held: 20
Enter classes attended: 12
Enter required attendance percentage: 75
Current attendance: 60.00%
Additional classes to attend: 12

Process finished with exit code 0
```

```python
held = int(input("Enter total classes held: "))
attended = int(input("Enter classes attended: "))
target = float(input("Enter required attendance percentage: "))

if held < 0 or attended < 0 or attended > held or target < 0 or target > 100:
    print("Invalid input.")
else:
    if held == 0:
        current = 0
    else:
        current = attended / held * 100
extra = 0
while (attended + extra) / (held + extra) * 100 < target:
    extra += 1
print(f"Current attendance: {current:.2f}%")
print("Additional classes to attend:", extra)

```

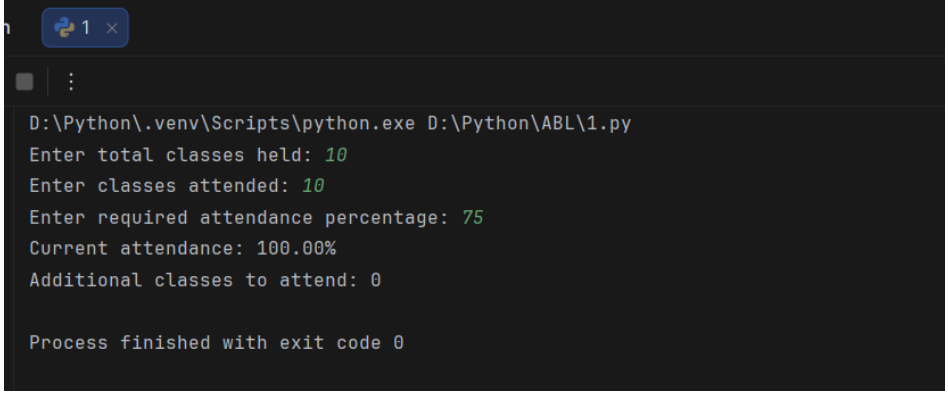
Run 1

```
D:\Python\.venv\Scripts\python.exe D:\Python\ABL\1.py
Enter total classes held: 40
Enter classes attended: 32
Enter required attendance percentage: 75
Current attendance: 80.00%
Additional classes to attend: 0

Process finished with exit code 0
```

```python
held = int(input("Enter total classes held: "))
attended = int(input("Enter classes attended: "))
target = float(input("Enter required attendance percentage: "))

if held < 0 or attended < 0 or attended > held or target < 0 or target > 100:
    print("Invalid input.")
else:
    if held == 0:
        current = 0
    else:
        current = attended / held * 100
extra = 0
while (attended + extra) / (held + extra) * 100 < target:
    extra += 1
print(f"Current attendance: {current:.2f}%")
print("Additional classes to attend:", extra)
```



Conclusion

The program calculates attendance and uses repetition to find the minimum number of future classes required to reach the target.

2.

ATM Transaction Sequence Simulation: Process a sequence of deposits and withdrawals, maintain a simulated balance, enforce withdrawal limits, reject invalid transactions, and produce a transaction summary.

Sol:

Problem statement

Develop a Python program that processes a sequence of ATM deposits and withdrawals, updates the account balance, rejects invalid transactions, and displays a transaction summary.

Objectives

- Perform deposits and withdrawals.
- Maintain the current balance.
- Reject invalid or unaffordable transactions.
- Count successful and rejected transactions.

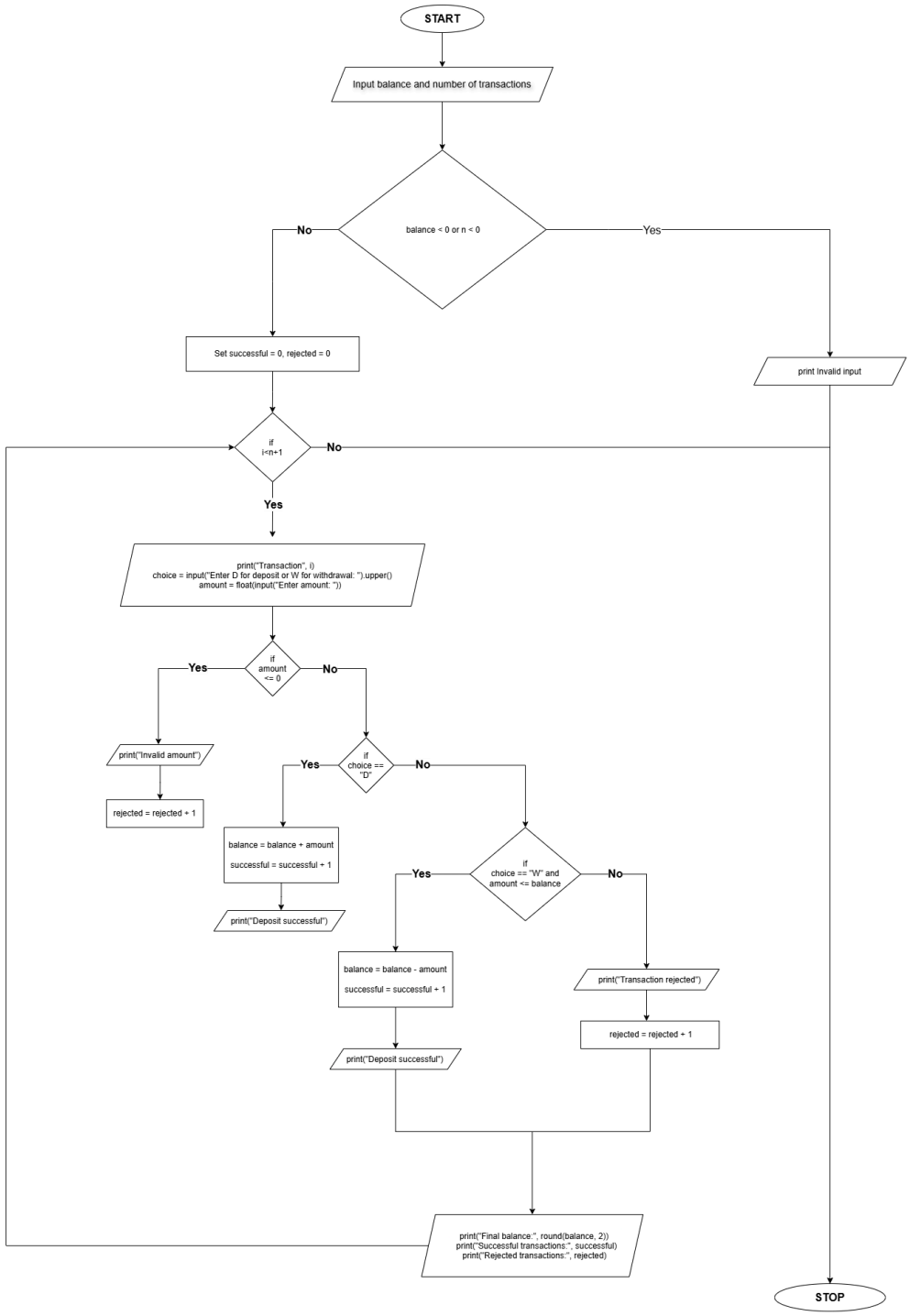
Assumptions

- The starting balance cannot be negative.
- The transaction type is D for deposit or W for withdrawal.
- Each transaction amount must be positive.
- Withdrawals cannot exceed the available balance.

Algorithm

1. Start.
2. Input the starting balance and number of transactions.
3. Set successful and rejected transaction counts to zero.
4. Repeat for each transaction.
5. Input the transaction type and amount.
6. If the transaction is valid, update the balance and successful count.
7. Otherwise, increase the rejected count.
8. Display the final balance and transaction counts.
9. Stop.

Flowchart



Pseudocode

START

INPUT balance, n

SET successful = 0

SET rejected = 0

REPEAT n times

```
    INPUT type, amount
    IF amount <= 0
        rejected = rejected + 1
    ELSE IF type = "D"
        balance = balance + amount
        successful = successful + 1
    ELSE IF type = "W" AND amount <= balance
        balance = balance - amount
        successful = successful + 1
    ELSE
        rejected = rejected + 1
PRINT final balance
PRINT successful transactions
PRINT rejected transactions
END
```

Source Code

```
balance = float(input("Enter starting balance: "))
n = int(input("How many transactions? "))
if balance < 0 or n < 0:
    print("Invalid input.")
else:
    successful = 0
    rejected = 0
    for i in range(1, n + 1):
        print("Transaction", i)
        choice = input("Enter D for deposit or W for withdrawal: ").upper()
        amount = float(input("Enter amount: "))
        if amount <= 0:
            print("Invalid amount")
            rejected = rejected + 1
        elif choice == "D":
            balance = balance + amount
            successful = successful + 1
            print("Deposit successful")
        elif choice == "W" and amount <= balance:
            balance = balance - amount
            successful = successful + 1
            print("Withdrawal successful")
        else:
```

```
        print("Transaction rejected")
        rejected = rejected + 1
print("Final balance:", round(balance, 2))
print("Successful transactions:", successful)
print("Rejected transactions:", rejected)
```

Test Cases

1. Enter starting balance: 1000
   How many transactions? 2
   Transaction 1
   Enter D for deposit or W for withdrawal: D
   Enter amount: 500
   Transaction 2
   Enter D for deposit or W for withdrawal: W
   Enter amount: 200
   Output:
   Transaction 1
   Deposit successful
   Transaction 2
   Withdrawal successful
   Final balance: 1300.0
   Successful transactions: 2
   Rejected transactions: 0
2. Enter starting balance: 500
   How many transactions? 2
   Transaction 1
   Enter D for deposit or W for withdrawal: W
   Enter amount: 700
   Transaction 2
   Enter D for deposit or W for withdrawal: D
   Enter amount: 200
   Output
   Transaction 1
   Transaction rejected
   Transaction 2
   Deposit successful
   Final balance: 700.0
   Successful transactions: 1
   Rejected transactions: 1

Output

```python
balance = float(input("Enter starting balance: "))
n = int(input("How many transactions? "))
if balance < 0 or n < 0:
    print("Invalid input.")
else:
    successful = 0
    rejected = 0
    for i in range(1, n + 1):
        print("Transaction", i)
        choice = input("Enter D for deposit or W for withdrawal: ").upper()
        amount = float(input("Enter amount: "))
        if amount <= 0:
            print("Invalid amount")
            rejected = rejected + 1
        elif choice == "D":
            balance = balance + amount
            successful = successful + 1
            print("Deposit successful")
        elif choice == "W" and amount <= balance:
            balance = balance - amount
            successful = successful + 1
            print("Withdrawal successful")
        else:
            print("Transaction rejected")
            rejected = rejected + 1
    print("Final balance:", round(balance, 2))
    print("Successful transactions:", successful)
    print("Rejected transactions:", rejected)
```

```
Run  1
D:\Python\.venv\Scripts\python.exe D:\Python\ABL\1.py
Enter starting balance: 500
How many transactions? 2
Transaction 1
Enter D for deposit or W for withdrawal: w
Enter amount: 700
Transaction rejected
Transaction 2
Enter D for deposit or W for withdrawal: d
Enter amount: 200
Deposit successful
Final balance: 700.0
Successful transactions: 1
Rejected transactions: 1

Process finished with exit code 0
```

```python
balance = float(input("Enter starting balance: "))
n = int(input("How many transactions? "))
if balance < 0 or n < 0:
    print("Invalid input.")
else:
    successful = 0
    rejected = 0
    for i in range(1, n + 1):
        print("Transaction", i)
        choice = input("Enter D for deposit or W for withdrawal: ").upper()
        amount = float(input("Enter amount: "))
        if amount <= 0:
            print("Invalid amount")
            rejected = rejected + 1
        elif choice == "D":
            balance = balance + amount
            successful = successful + 1
            print("Deposit successful")
        elif choice == "W" and amount <= balance:
            balance = balance - amount
            successful = successful + 1
            print("Withdrawal successful")
        else:
            print("Transaction rejected")
            rejected = rejected + 1
    print("Final balance:", round(balance, 2))
    print("Successful transactions:", successful)
    print("Rejected transactions:", rejected)
```

```
Run  1
D:\Python\.venv\Scripts\python.exe D:\Python\ABL\1.py
Enter starting balance: 1000
How many transactions? 2
Transaction 1
Enter D for deposit or W for withdrawal: d
Enter amount: 500
Deposit successful
Transaction 2
Enter D for deposit or W for withdrawal: w
Enter amount: 200
Withdrawal successful
Final balance: 1300.0
Successful transactions: 2
Rejected transactions: 0

Process finished with exit code 0
```

Conclusion

The program simulates ATM transactions using loops, conditions, and arithmetic operations. It also tracks successful and rejected transactions.

3.

Student Performance Trend Analysis: Compare assessment scores across multiple examinations, calculate improvements and declines, identify consistently weak subjects, and

classify students according to defined criteria.

Sol:

Problem statement

Compare assessment scores across examinations, calculate improvements and declines, identify weak subjects, and classify students.

Objectives

- Calculate score changes.
- Identify weak subjects.
- Classify performance.

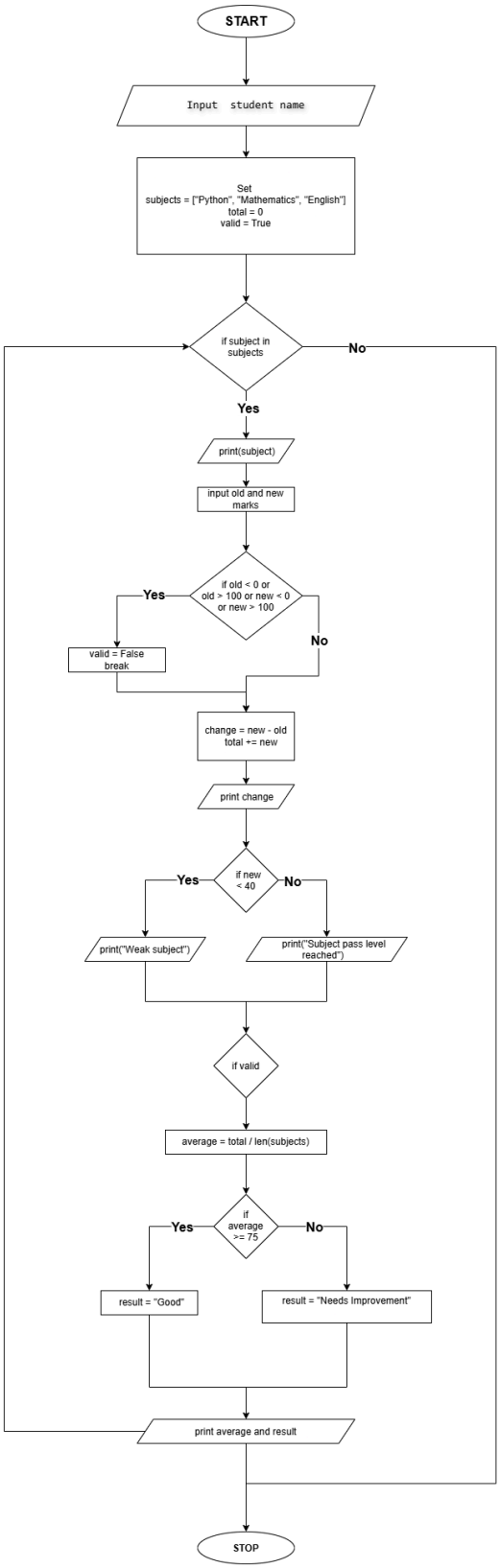
Assumptions

- Three subjects and two examinations are considered.
- Scores range from 0 to 100.
- A subject is weak if the latest score is below 40.
- Average score of 75 or more is classified as Good; otherwise, Needs Improvement.

Algorithm

1. Input the student's name.
2. Input earlier and latest scores for three subjects.
3. Calculate the average latest score.
4. Calculate each subject's score change.
5. Identify weak subjects.
6. Classify overall performance.
7. Display the report.

Flowchart


START
Input student name
Set
subjects = ["Python", "Mathematics", "English"]
total = 0
valid = True
if subject in subjects
No
Yes
print(subject)
input old and new marks
if old < 0 or old > 100 or new < 0 or new > 100
Yes
No
valid = False
break
change = new - old
total += new
print change
if new < 40
Yes
No
print("Weak subject")
print("Subject pass level reached")
if valid
average = total / len(subjects)
if average >= 75
Yes
No
result = "Good"
result = "Needs Improvement"
print average and result
STOP

Pseudocode

```
START
INPUT student name
FOR each subject
   INPUT earlier and latest marks
   CALCULATE change
   CHECK weak subject
END FOR
CALCULATE average
IF average >= 75
   classification = Good
ELSE
   classification = Needs Improvement
END IF
DISPLAY report
STOP
```

Source Code

```
name = input("Student name: ")
subjects = ["Python", "Mathematics", "English"]
total = 0
valid = True
for subject in subjects:
   print(subject)
   old = int(input("Earlier marks: "))
   new = int(input("Latest marks: "))
   if old < 0 or old > 100 or new < 0 or new > 100:
      valid = False
      print("Invalid marks")
      break
   change = new - old
   total += new
   print("Change:", change)
   if new < 40:
      print("Weak subject")
   else:
      print("Subject pass level reached")

if valid==True:
```

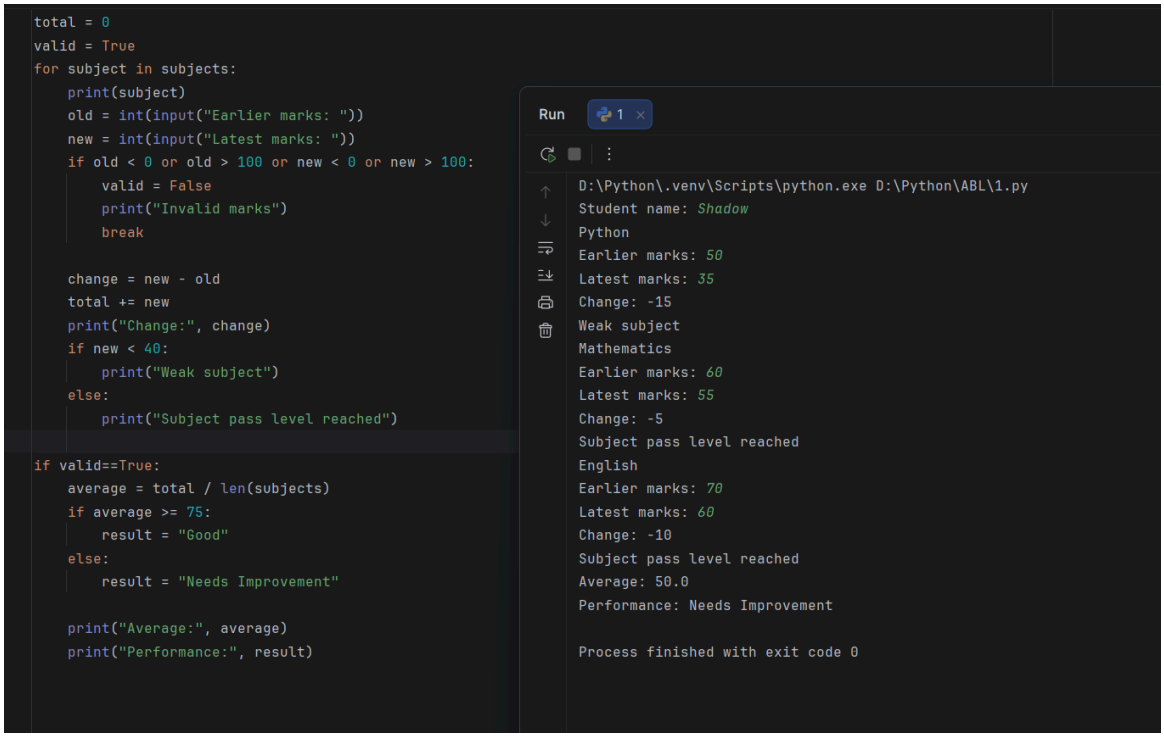
```
average = total / len(subjects)
if average >= 75:
    result = "Good"
else:
    result = "Needs Improvement"
print("Average:", average)
print("Performance:", result)
```

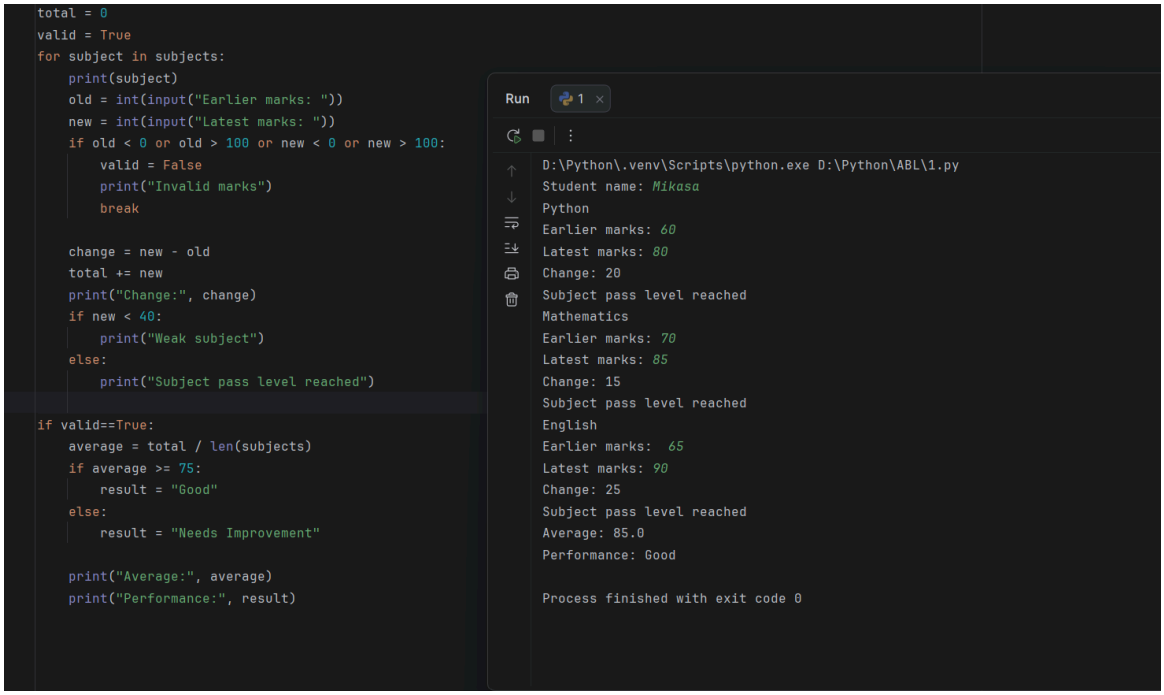
Test Cases

1. Student name:Mikasa
   Python
   Earlier marks: 60
   Latest marks: 80
   Mathematics
   Earlier marks: 70
   Latest marks: 85
   English
   Earlier marks: 65
   Latest marks: 90
   Output
   Python
   Change: 20
   Subject pass level reached
   Mathematics
   Change: 15
   Subject pass level reached
   English
   Change: 25
   Subject pass level reached
   Average: 85.0
   Performance: Good
2. Student name:Shadow
   Python
   Earlier marks: 50
   Latest marks: 35
   Mathematics
   Earlier marks: 60
   Latest marks: 55
   English

Earlier marks: 70
Latest marks: 60
Output
Python
Change: -15
Weak subject
Mathematics
Change: -5
Subject pass level reached
English
Change: -10
Subject pass level reached
Average: 50.0
Performance: Needs Improvement

Output





Conclusion

The program analyses student performance across examinations and identifies improvements, declines and weak subjects.

4.

Repeated Transaction Detection: Process transaction identifiers and amounts in a list, detect duplicate transactions, identify threshold violations, and generate a list of suspicious entries.

Sol:

Problem statement

Develop a Python program to store transaction identifiers and amounts in lists, detect repeated transaction IDs or amounts exceeding a specified limit, and display all suspicious transactions.

Objectives

- Read and store transaction identifiers and amounts in lists.
- Detect duplicate transaction identifiers and high-value threshold violations.
- Generate and display a list of suspicious entries.

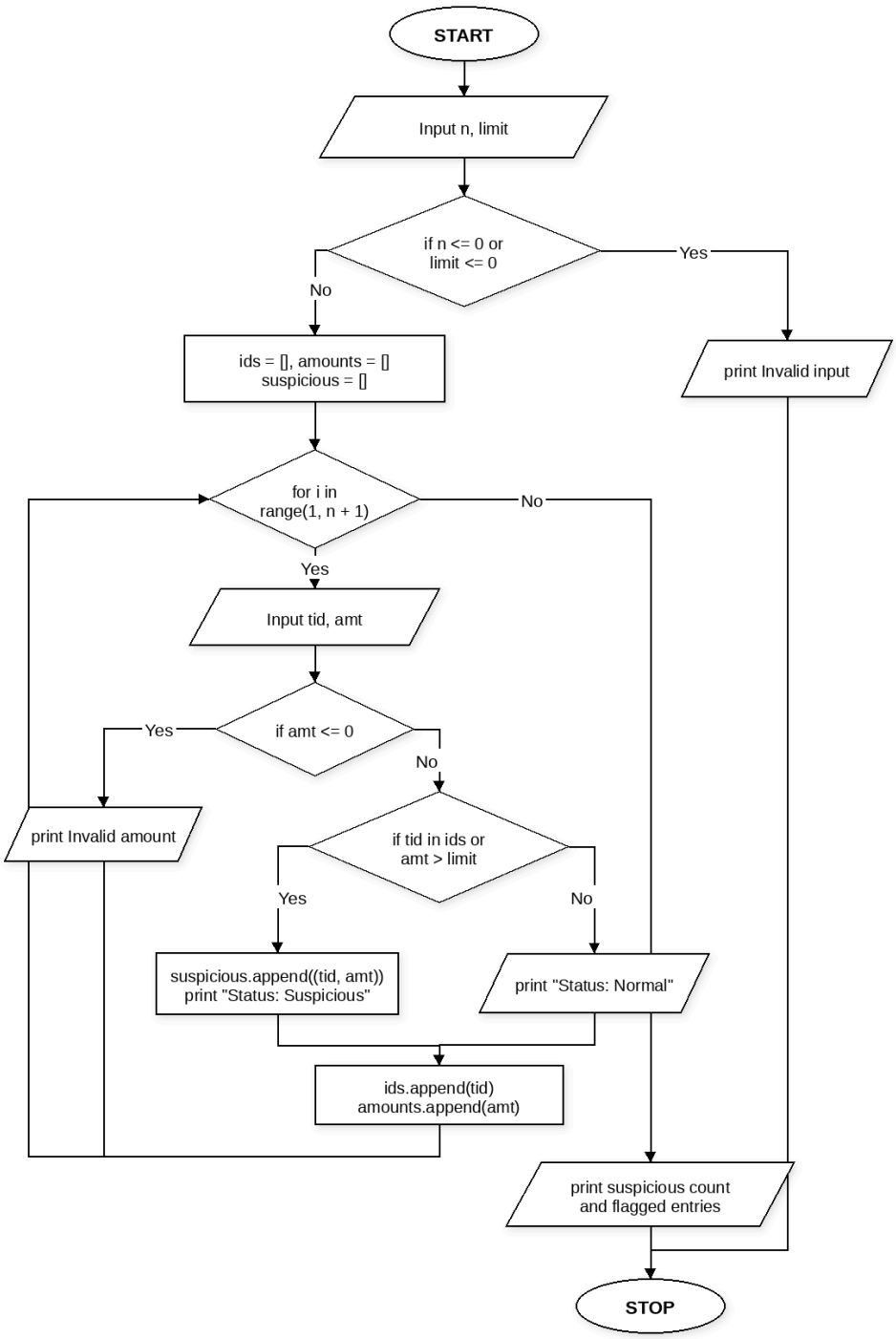
Assumptions

- The number of transactions and threshold limit must be positive.
- Each transaction amount must be greater than zero.
- A transaction is flagged if its ID has appeared earlier or its amount exceeds the threshold.

Algorithm

1. Start.
2. Read the number of transactions and the high-value threshold.
3. Validate the inputs.
4. Initialize empty lists for seen IDs, amounts, and suspicious entries.
5. For each transaction, read the transaction ID and amount.
6. Check if the ID is already in the IDs list or if the amount exceeds the limit.
7. If either condition holds, append the transaction tuple to the suspicious list.
8. Display the total count and details of suspicious entries.
9. Stop.

Flowchart



Pseudocode

START
INPUT n, limit
IF n <= 0 OR limit <= 0
    PRINT "Invalid input."
ELSE

```
    SET ids = [], amounts = [], suspicious = []
    FOR i from 1 to n
        INPUT tid, amt
        IF amt <= 0
            PRINT "Invalid amount"
        ELSE
            IF tid in ids OR amt > limit
                APPEND (tid, amt) to suspicious
                PRINT "Status: Suspicious"
            ELSE
                PRINT "Status: Normal"
            APPEND tid to ids
            APPEND amt to amounts
    PRINT length of suspicious
    FOR each (t_id, t_amt) in suspicious
        PRINT t_id, t_amt
STOP
```

Source Code

```
n = int(input("Enter number of transactions: "))
limit = float(input("Enter high-value threshold: "))
if n <= 0 or limit <= 0:
    print("Invalid input.")
else:
    ids = []
    amounts = []
    suspicious = []
    for i in range(1, n + 1):
        tid = input("Transaction " + str(i) + " ID: ")
        amt = float(input("Transaction " + str(i) + " amount: "))
        if amt <= 0:
            print("Invalid amount")
        else:
            if tid in ids or amt > limit:
                suspicious.append((tid, amt))
                print("Status: Suspicious")
            else:
                print("Status: Normal")
            ids.append(tid)
            amounts.append(amt)
```

```
print("Suspicious count:", len(suspicious))
for item in suspicious:
    t_id, t_amt = item
    print("Flagged:", t_id, "Rs.", t_amt)
```

Test Cases

1. Enter number of transactions: 3
   Enter high-value threshold: 5000
   Transaction 1 ID: TX101
   Transaction 1 amount: 1200
   Transaction 2 ID: TX101
   Transaction 2 amount: 1200
   Transaction 3 ID: TX102
   Transaction 3 amount: 6500
   Output:
   Status: Normal
   Status: Suspicious
   Status: Suspicious
   Suspicious count: 2
   Flagged: TX101 Rs. 1200.0
   Flagged: TX102 Rs. 6500.0
2. Enter number of transactions: 2
   Enter high-value threshold: 3000
   Transaction 1 ID: TX201
   Transaction 1 amount: 800
   Transaction 2 ID: TX202
   Transaction 2 amount: 1500
   Output:
   Status: Normal
   Status: Normal
   Suspicious count: 0

Output

```python
n = int(input("Enter number of transactions: "))
limit = float(input("Enter high-value threshold: "))
if n <= 0 or limit <= 0:
    print("Invalid input.")
else:
    ids = []
    amounts = []
    suspicious = []
    for i in range(1, n + 1):
        tid = input("Transaction " + str(i) + " ID: ")
        amt = float(input("Transaction " + str(i) + " amount: "
        if amt <= 0:
            print("Invalid amount")
        else:
            if tid in ids or amt > limit:
                suspicious.append((tid, amt))
                print("Status: Suspicious")
            else:
                print("Status: Normal")
            ids.append(tid)
            amounts.append(amt)
    print("Suspicious count:", len(suspicious))
    for item in suspicious:
        t_id, t_amt = item
        print("Flagged:", t_id, "Rs.", t_amt)
```

```
Run  1
D:\Python\.venv\Scripts\python.exe D:\Python\ABL\1.py
Enter number of transactions: 3
Enter high-value threshold: 5000
Transaction 1 ID: TX101
Transaction 1 amount: 1200
Status: Normal
Transaction 2 ID: TX101
Transaction 2 amount: 1200
Status: Suspicious
Transaction 3 ID: TX102
Transaction 3 amount: 6500
Status: Suspicious
Suspicious count: 2
Flagged: TX101 Rs. 1200.0
Flagged: TX102 Rs. 6500.0

Process finished with exit code 0
```

```python
n = int(input("Enter number of transactions: "))
limit = float(input("Enter high-value threshold: "))
if n <= 0 or limit <= 0:
    print("Invalid input.")
else:
    ids = []
    amounts = []
    suspicious = []
    for i in range(1, n + 1):
        tid = input("Transaction " + str(i) + " ID: ")
        amt = float(input("Transaction " + str(i) + " amount: "
        if amt <= 0:
            print("Invalid amount")
        else:
            if tid in ids or amt > limit:
                suspicious.append((tid, amt))
                print("Status: Suspicious")
            else:
                print("Status: Normal")
            ids.append(tid)
            amounts.append(amt)
    print("Suspicious count:", len(suspicious))
    for item in suspicious:
        t_id, t_amt = item
        print("Flagged:", t_id, "Rs.", t_amt)
```

```
Run  1
D:\Python\.venv\Scripts\python.exe D:\Python\ABL\1.py
Enter number of transactions: 2
Enter high-value threshold: 3000
Transaction 1 ID: TX201
Transaction 1 amount: 800
Status: Normal
Transaction 2 ID: TX202
Transaction 2 amount: 1500
Status: Normal
Suspicious count: 0

Process finished with exit code 0
```

Conclusion

The program processes transaction records using lists and tuples to detect duplicate identifiers and threshold violations.

5.

Food Waste Monitoring System: Compare prepared meals, meals served, and leftovers across several days; calculate wastage percentages and identify days exceeding acceptable limits.

Sol:

Problem statement

Compare the number of meals prepared, meals served, and leftovers over several days; calculate waste percentages and identify days that exceed an acceptable limit.

Objectives

- Calculate leftover meals.
- Calculate waste percentage for each day.
- Identify days above the supplied waste limit.

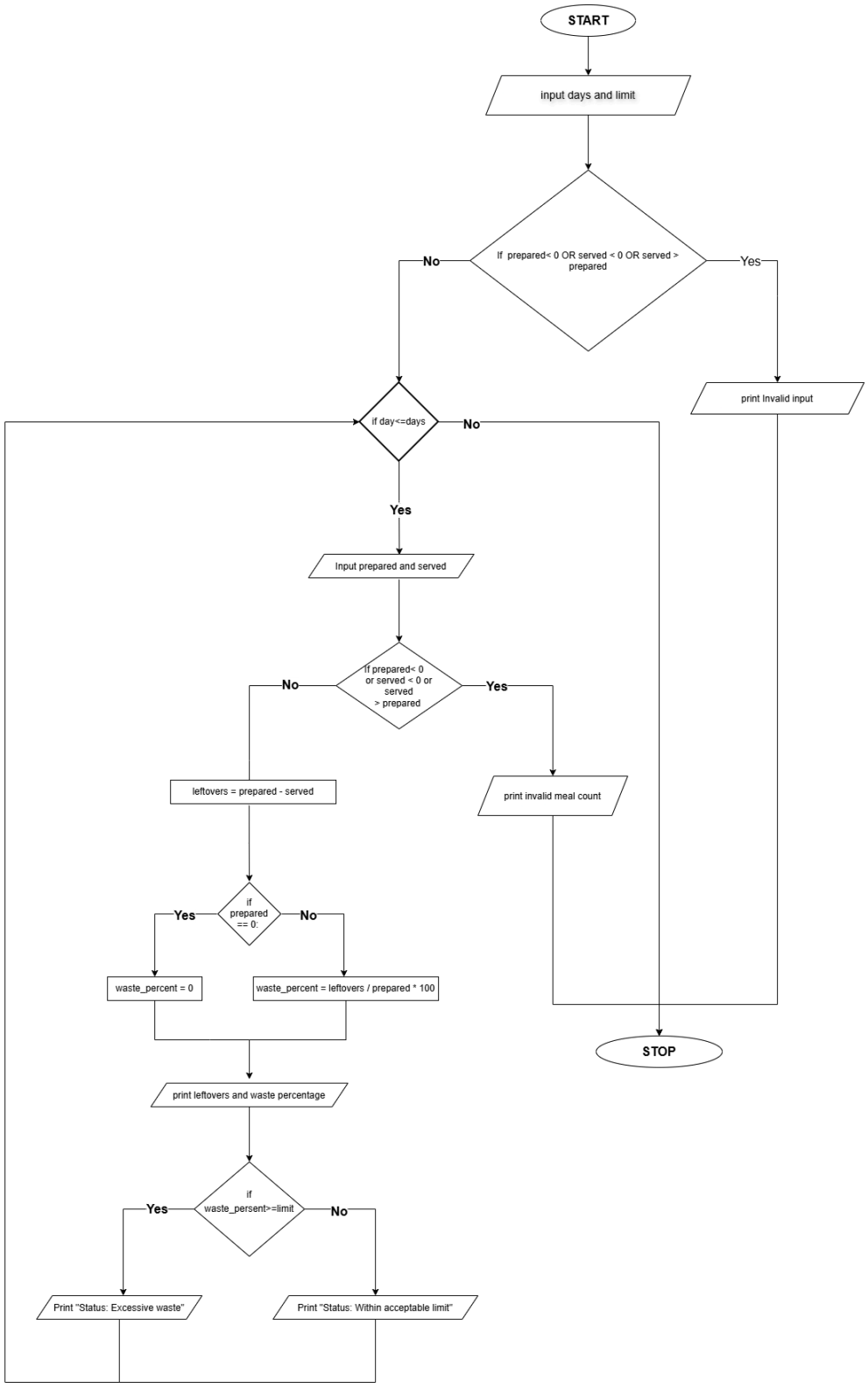
Assumptions

- Prepared and served meals are non-negative integers.
- Meals served cannot exceed meals prepared.
- Waste percentage is leftovers divided by meals prepared, multiplied by 100.

Algorithm

1. Read the number of days and acceptable waste percentage.
2. For each day, read meals prepared and served.
3. Validate the values.
4. Calculate leftovers and waste percentage.
5. Classify the day as within limit or excessive waste.
6. Display each day's result.

Flowchart



Pseudocode

START

```
INPUT days, limit
IF days <= 0 OR limit < 0 OR limit > 100
   PRINT "Invalid input"
ELSE
   FOR each day
      INPUT prepared, served
      IF prepared < 0 OR served < 0 OR served > prepared
         PRINT "Invalid meal counts"
      ELSE
         leftovers = prepared - served
         IF prepared = 0
            Waste_persent= 0
         ELSE
            Waste_persent= leftovers / prepared * 100
         PRINT leftovers, waste_persent
         IF waste_persent> limit
            PRINT "Excessive waste"
         ELSE
            PRINT "Within acceptable limit"
STOP
```

Source Code

```
days = int(input("Enter number of days: "))
limit = float(input("Enter acceptable waste percentage: "))

if days <= 0 or limit < 0 or limit > 100:
   print("Invalid input.")
else:
   for day in range(1, days + 1):
      prepared = int(input("Meals prepared on day " + str(day) + ": "))
      served = int(input("Meals served on day " + str(day) + ": "))

      if prepared < 0 or served < 0 or served > prepared:
         print("Invalid meal counts.")
      else:
         leftovers = prepared - served
         if prepared == 0:
            waste_percent = 0
         else:
            waste_percent = leftovers / prepared * 100
```

```
print("Leftover meals:", leftovers)
print("Waste percentage:", round(waste_percent, 2), "%")

if waste_percent > limit:
    print("Status: Excessive waste")
else:
    print("Status: Within acceptable limit")
```

Test Cases

1. Enter number of days: 1
   Enter acceptable waste percentage: 20
   Meals prepared on day 1: 100
   Meals served on day 1: 90
   Output:
   Leftover meals: 10
   Waste percentage: 10.0 %
   Status: Within acceptable limit
2. Enter number of days: 1
   Enter acceptable waste percentage: 15
   Meals prepared on day 1: 200
   Meals served on day 1: 150
   Output:
   Leftover meals: 50
   Waste percentage: 25.0 %
   Status: Excessive waste

Output

```python
days = int(input("Enter number of days: "))
limit = float(input("Enter acceptable waste percentage: "))

if days <= 0 or limit < 0 or limit > 100:
    print("Invalid input.")
else:
    for day in range(1, days + 1):
        prepared = int(input("Meals prepared on day " + str(day) + ": "))
        served = int(input("Meals served on day " + str(day) + ": "))

        if prepared < 0 or served < 0 or served > prepared:
            print("Invalid meal counts.")
        else:
            leftovers = prepared - served

            if prepared == 0:
                waste_percent = 0
            else:
                waste_percent = leftovers / prepared * 100

            print("Leftover meals:", leftovers)
            print("Waste percentage:", round(waste_percent, 2), "%")

            if waste_percent > limit:
                print("Status: Excessive waste")
            else:
                print("Status: Within acceptable limit")
```

```
Run  1
Enter acceptable waste percentage: 15
Meals prepared on day 1: 200
Meals served on day 1: 150
Leftover meals: 50
Waste percentage: 25.0 %
Status: Excessive waste

Process finished with exit code 0
```

```python
days = int(input("Enter number of days: "))
limit = float(input("Enter acceptable waste percentage: "))

if days <= 0 or limit < 0 or limit > 100:
    print("Invalid input.")
else:
    for day in range(1, days + 1):
        prepared = int(input("Meals prepared on day " + str(day) + ": "))
        served = int(input("Meals served on day " + str(day) + ": "))

        if prepared < 0 or served < 0 or served > prepared:
            print("Invalid meal counts.")
        else:
            leftovers = prepared - served

            if prepared == 0:
                waste_percent = 0
            else:
                waste_percent = leftovers / prepared * 100

            print("Leftover meals:", leftovers)
            print("Waste percentage:", round(waste_percent, 2), "%")

            if waste_percent > limit:
                print("Status: Excessive waste")
            else:
                print("Status: Within acceptable limit")
```

```
Run  1
D:\Python\.venv\Scripts\python.exe D:\Python\ABL\1.py
Enter number of days: 1
Enter acceptable waste percentage: 20
Meals prepared on day 1: 100
Meals served on day 1: 90
Leftover meals: 10
Waste percentage: 10.0 %
Status: Within acceptable limit
```

Conclusion

The program uses arithmetic and conditions to monitor leftover food and flag days with high waste.

6.

Student Timetable Conflict Detection: Compare course schedules represented by strings and tuples, detect overlapping periods, identify unavailable rooms, and validate a proposed timetable.

Sol:

Problem statement

Develop a Python program that stores proposed course slots as tuples of course code, day, period, and room, checks for unavailable rooms and overlapping time periods, and validates the timetable.

Objectives

- Represent course schedule entries using strings and tuples.
- Identify unavailable classroom assignments.
- Detect overlapping periods on the same day.
- Display confirmed slots and total conflicts.

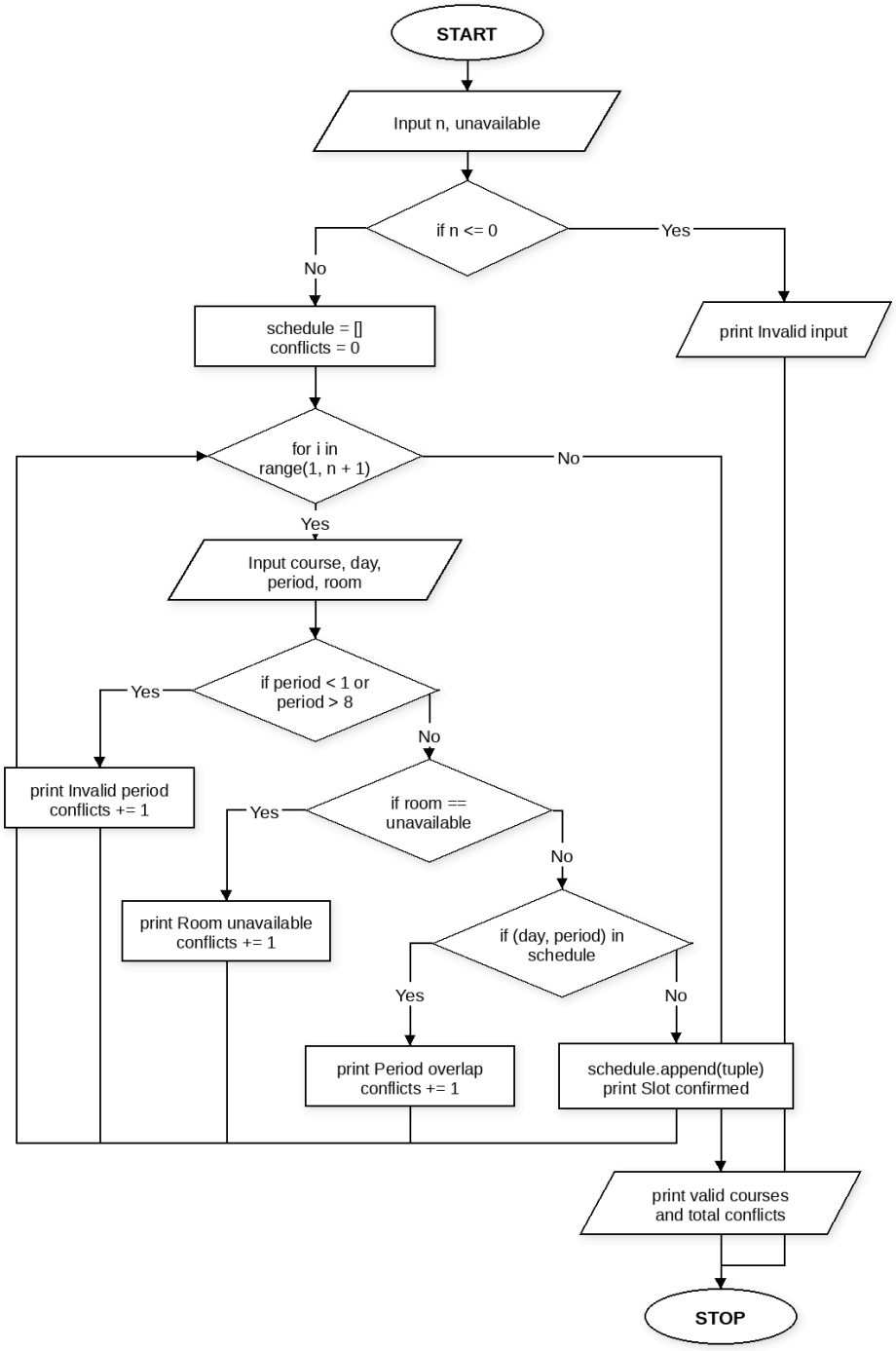
Assumptions

- Periods are numbered from 1 to 8 within a day.
- Two courses conflict if they are scheduled on the same day and period.
- Room and day names are compared in uppercase format.

Algorithm

1. Start.
2. Input the number of courses and the unavailable room code.
3. Initialize an empty schedule list and set conflict count to zero.
4. For each course, input the course code, day, period, and room.
5. Validate the period range and check if the room matches the unavailable room.
6. Traverse the confirmed schedule tuples using unpacking to check if the same day and period already exist.
7. If a conflict exists, increment the conflict counter; otherwise, append the tuple to the schedule.
8. Display the number of valid courses and total conflicts.
9. Stop.

Flowchart



Pseudocode

START

INPUT n, unavailable

IF n <= 0

    PRINT "Invalid input."

ELSE

```
    SET schedule = [], conflicts = 0
    FOR i from 1 to n
        INPUT course, day, period, room
        IF period < 1 OR period > 8
            PRINT "Invalid period"
            conflicts = conflicts + 1
        ELSE IF room == unavailable
            PRINT "Alert: Room unavailable"
            conflicts = conflicts + 1
        ELSE
            SET clash = False
            FOR each (c_code, c_day, c_per, c_room) in schedule
                IF c_day == day AND c_per == period
                    clash = True
            IF clash == True
                PRINT "Conflict: Period overlap"
                conflicts = conflicts + 1
            ELSE
                APPEND (course, day, period, room) to schedule
                PRINT "Slot confirmed"
    PRINT length of schedule, conflicts
STOP
```

Source Code

```
n = int(input("Enter number of courses: "))
unavailable = input("Enter unavailable room: ").upper()
if n <= 0:
    print("Invalid input.")
else:
    schedule = []
    conflicts = 0
    for i in range(1, n + 1):
        course = input("Course " + str(i) + " code: ").upper()
        day = input("Day: ").upper()
        period = int(input("Period (1-8): "))
        room = input("Room: ").upper()
        if period < 1 or period > 8:
            print("Invalid period")
            conflicts += 1
        elif room == unavailable:
```

```
            print("Alert: Room unavailable")
            conflicts += 1
        else:
            clash = False
            for entry in schedule:
                c_code, c_day, c_per, c_room = entry
                if c_day == day and c_per == period:
                    clash = True
            if clash:
                print("Conflict: Period overlap")
                conflicts += 1
            else:
                schedule.append((course, day, period, room))
                print("Slot confirmed")
    print("Valid courses:", len(schedule))
    print("Total conflicts:", conflicts)
```

Test Cases

1. Enter number of courses: 2
   Enter unavailable room: R105
   Course 1 code: CS101
   Day: MON
   Period (1-8): 2
   Room: R101
   Course 2 code: MA102
   Day: MON
   Period (1-8): 2
   Room: R102
   Output:
   Slot confirmed
   Conflict: Period overlap
   Valid courses: 1
   Total conflicts: 1
2. Enter number of courses: 2
   Enter unavailable room: R105
   Course 1 code: EC101
   Day: TUE
   Period (1-8): 3
   Room: R105

Course 2 code: PH101

Day: WED

Period (1-8): 1

Room: R102

Output:

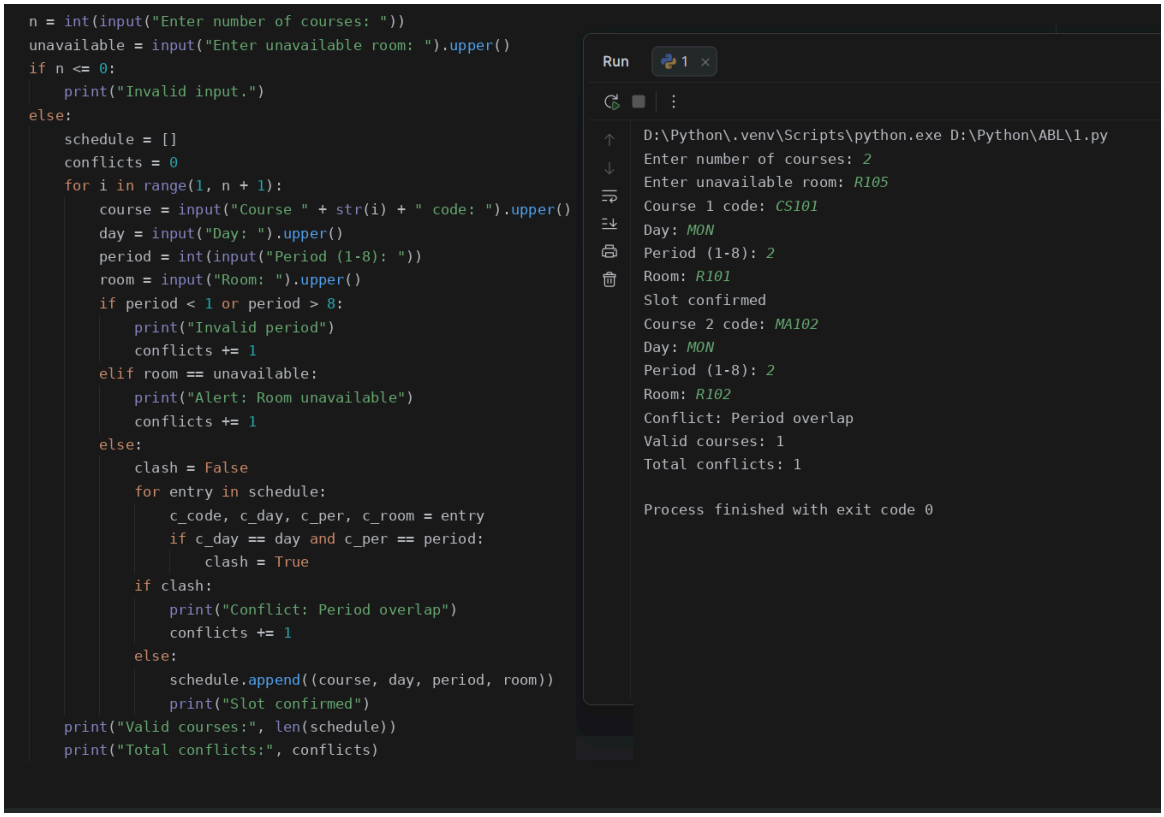
Alert: Room unavailable

Slot confirmed

Valid courses: 1

Total conflicts: 1

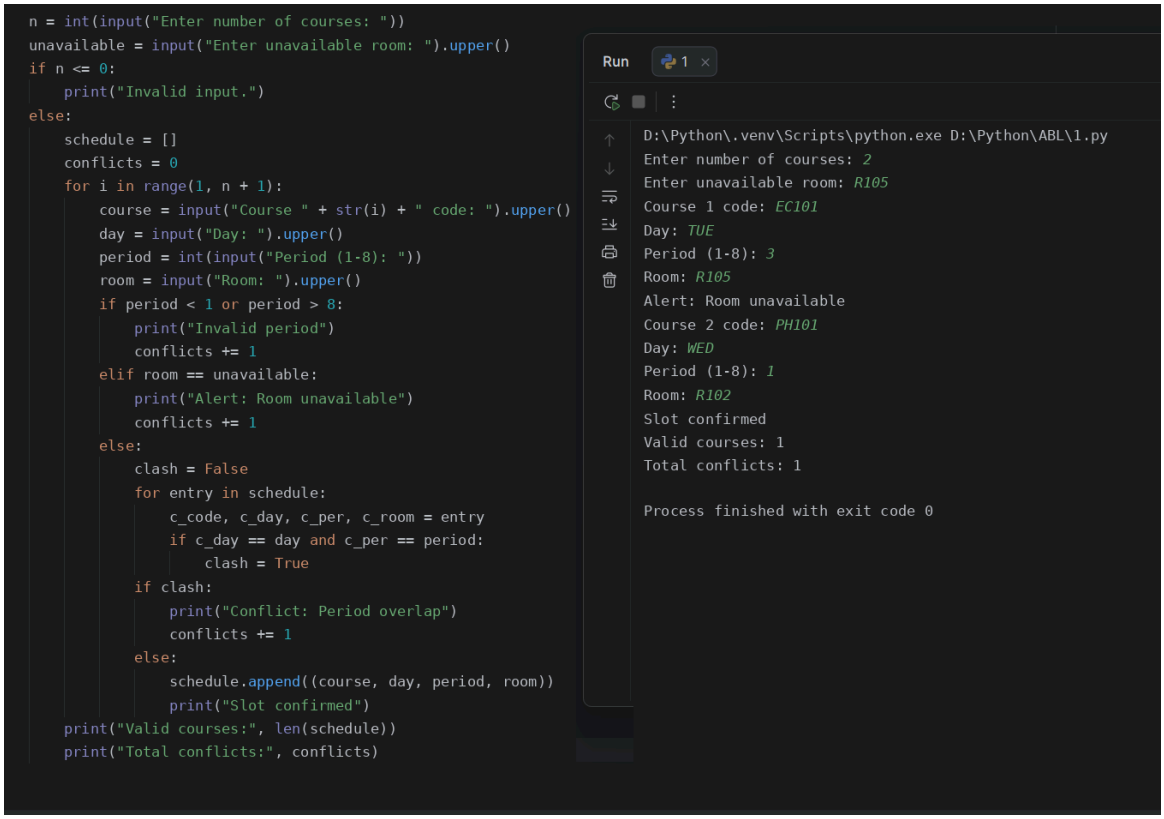
Output



```
n = int(input("Enter number of courses: "))
unavailable = input("Enter unavailable room: ").upper()
if n <= 0:
    print("Invalid input.")
else:
    schedule = []
    conflicts = 0
    for i in range(1, n + 1):
        course = input("Course " + str(i) + " code: ").upper()
        day = input("Day: ").upper()
        period = int(input("Period (1-8): "))
        room = input("Room: ").upper()
        if period < 1 or period > 8:
            print("Invalid period")
            conflicts += 1
        elif room == unavailable:
            print("Alert: Room unavailable")
            conflicts += 1
        else:
            clash = False
            for entry in schedule:
                c_code, c_day, c_per, c_room = entry
                if c_day == day and c_per == period:
                    clash = True
            if clash:
                print("Conflict: Period overlap")
                conflicts += 1
            else:
                schedule.append((course, day, period, room))
                print("Slot confirmed")
    print("Valid courses:", len(schedule))
    print("Total conflicts:", conflicts)
```

```
Run  1
D:\Python\.venv\Scripts\python.exe D:\Python\ABL\1.py
Enter number of courses: 2
Enter unavailable room: R105
Course 1 code: CS101
Day: MON
Period (1-8): 2
Room: R101
Slot confirmed
Course 2 code: MA102
Day: MON
Period (1-8): 2
Room: R102
Conflict: Period overlap
Valid courses: 1
Total conflicts: 1

Process finished with exit code 0
```



Conclusion

The program validates a student timetable using strings, tuples, and tuple unpacking to catch room restrictions and period clashes.

7.

Student Marks Matrix Analysis: Analyse a matrix of student marks across subjects, calculate row-wise and column-wise statistics, identify top performers, and detect subjects with low average performance.

Sol:

Problem statement

Develop a Python program to store student marks across multiple subjects in a nested list matrix, compute student totals and averages, find the top performer, and flag subjects with an average below 50.

Objectives

- Store student marks in a two-dimensional nested list.
- Calculate row-wise totals and averages for each student.

- Identify the student with the highest total score.
- Calculate column-wise averages to detect low-performing subjects.

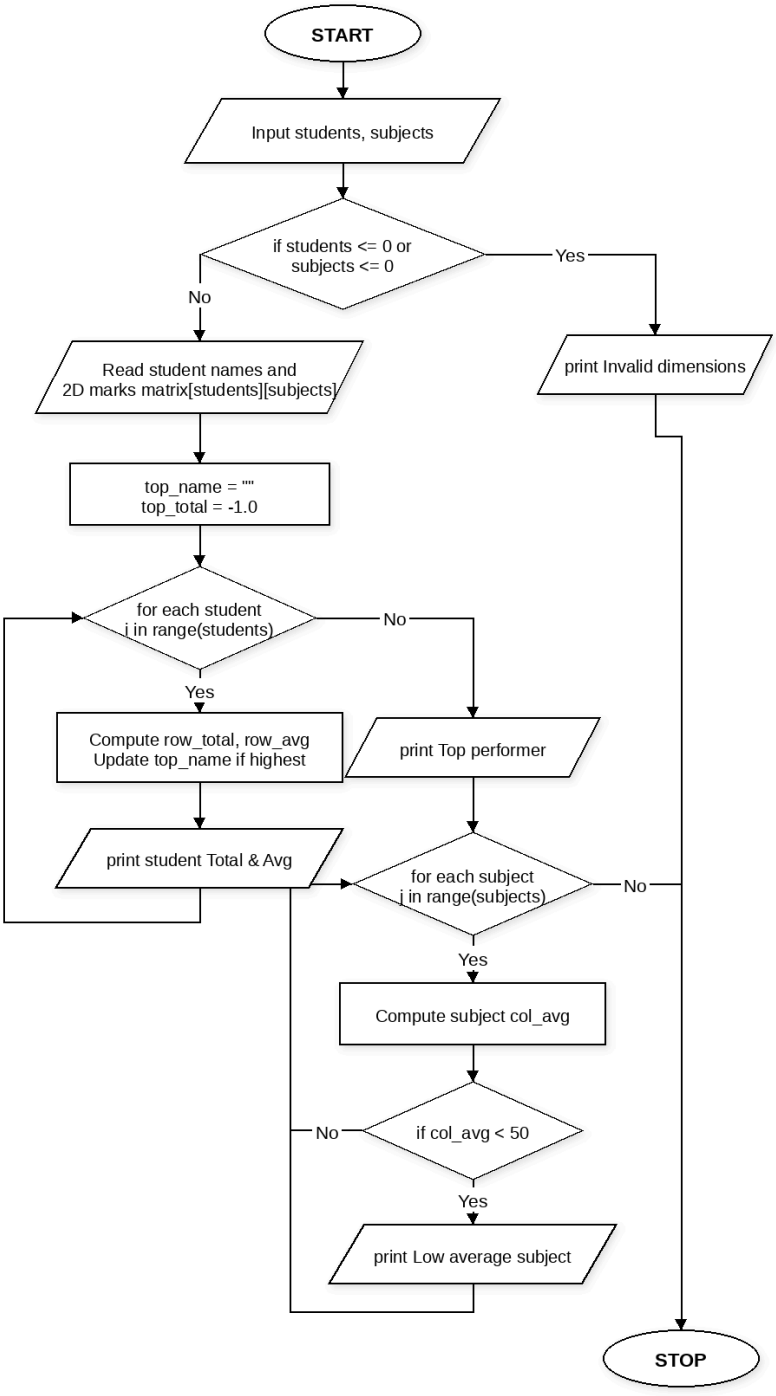
Assumptions

- The number of students and subjects are positive integers.
- Marks for each subject are between 0 and 100.
- A subject is flagged as low performing if its class average is below 50.

Algorithm

1. Start.
2. Read the number of students and number of subjects.
3. Validate that both dimensions are positive.
4. Use nested loops to read each student's name and subject marks into a 2D nested list.
5. Traverse each row to calculate the student's total and average, updating the top performer.
6. Traverse each column across all students to calculate subject averages.
7. Display student statistics, the top performer, and any low-average subjects.
8. Stop.

Flowchart



Pseudocode

START
INPUT students, subjects
IF students <= 0 OR subjects <= 0
    PRINT "Invalid dimensions."
ELSE

```
    SET matrix = [], names = []
    FOR i from 0 to students - 1
        INPUT student name and append to names
        SET row = []
        FOR j from 0 to subjects - 1
            INPUT mark and append to row
        APPEND row to matrix
    SET top_name = "", top_total = -1
    FOR i from 0 to students - 1
        CALCULATE row_total and row_avg
        PRINT names[i], row_total, row_avg
        IF row_total > top_total
            SET top_total = row_total, top_name = names[i]
    PRINT top_name
    FOR j from 0 to subjects - 1
        CALCULATE col_avg across all students
        IF col_avg < 50
            PRINT low average subject j + 1
STOP
```

Source Code

```
students = int(input("Enter number of students: "))
subjects = int(input("Enter number of subjects: "))
if students <= 0 or subjects <= 0:
    print("Invalid dimensions.")
else:
    matrix = []
    names = []
    for i in range(students):
        name = input("Student " + str(i + 1) + " name: ")
        names.append(name)
        row = []
        for j in range(subjects):
            m = float(input(" Subject " + str(j + 1) + " marks: "))
            row.append(m)
        matrix.append(row)
    top_name = ""
    top_total = -1.0
    for i in range(students):
        row_total = 0.0
```

```
    for m in matrix[i]:
        row_total += m
    row_avg = row_total / subjects
    print(names[i], "Total:", row_total, "Avg:", round(row_avg, 2))
    if row_total > top_total:
        top_total = row_total
        top_name = names[i]
print("Top performer:", top_name)
for j in range(subjects):
    col_total = 0.0
    for i in range(students):
        col_total += matrix[i][j]
    col_avg = col_total / students
    if col_avg < 50:
        print("Low average in Subject", j + 1, ":", round(col_avg, 2))
```

Test Cases

1. Enter number of students: 2
   Enter number of subjects: 3
   Student 1 name: Arjun
   Subject 1 marks: 80
   Subject 2 marks: 40
   Subject 3 marks: 75
   Student 2 name: Priya
   Subject 1 marks: 90
   Subject 2 marks: 50
   Subject 3 marks: 85
   Output:
   Arjun Total: 195.0 Avg: 65.0
   Priya Total: 225.0 Avg: 75.0
   Top performer: Priya
   Low average in Subject 2 : 45.0
2. Enter number of students: 2
   Enter number of subjects: 2
   Student 1 name: Kiran
   Subject 1 marks: 88
   Subject 2 marks: 92
   Student 2 name: Meera
   Subject 1 marks: 76

Subject 2 marks: 84

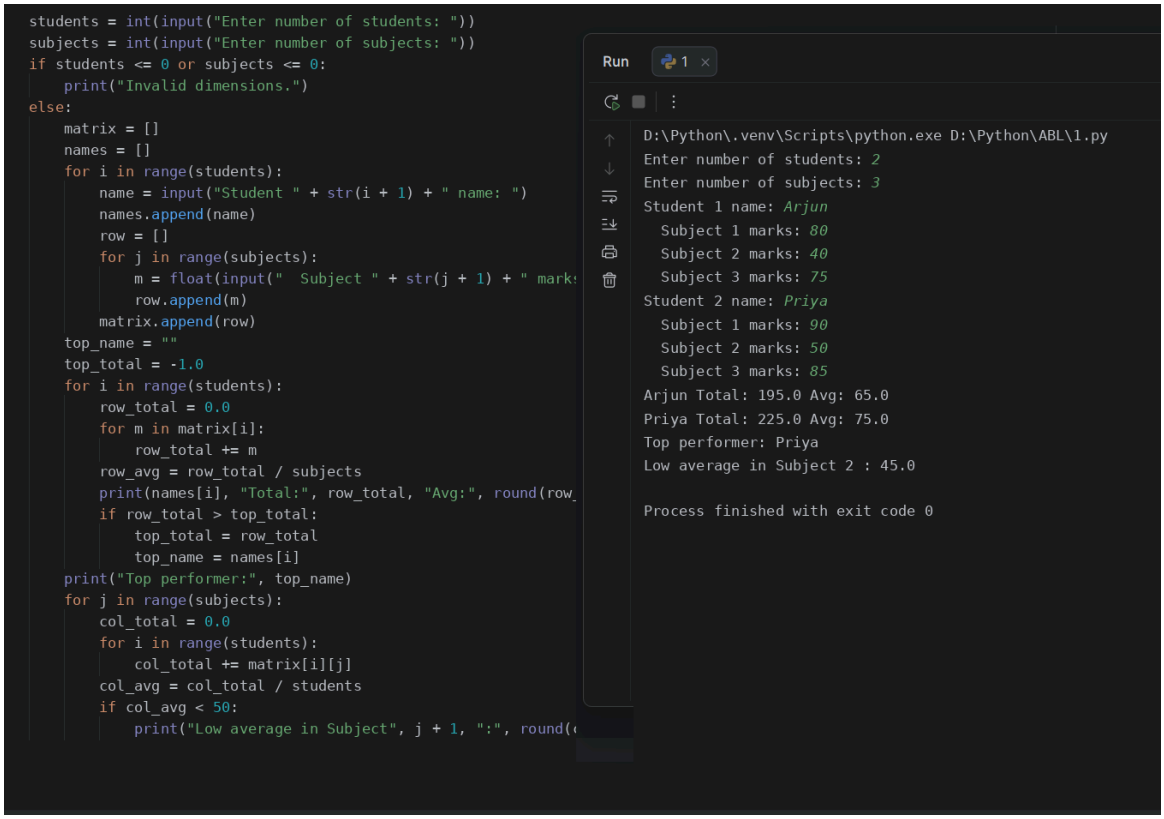
Output:

Kiran Total: 180.0 Avg: 90.0

Meera Total: 160.0 Avg: 80.0

Top performer: Kiran

Output

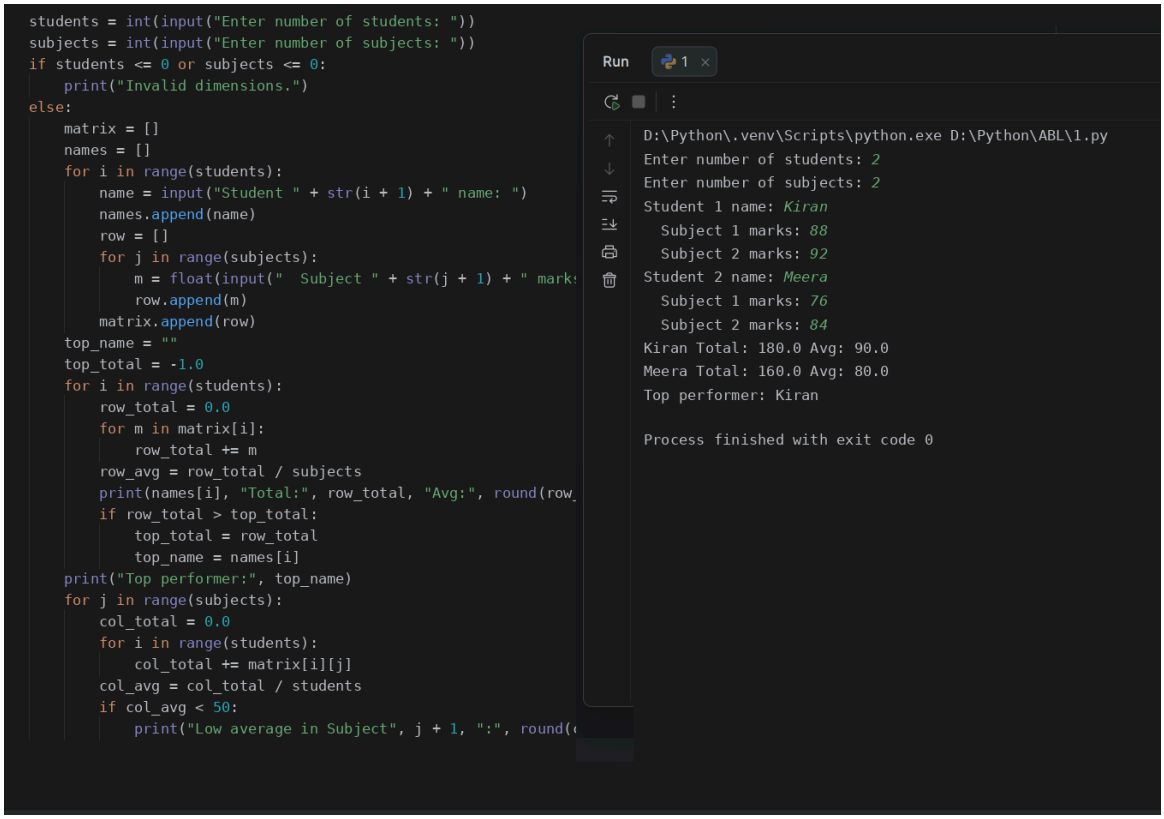


```
students = int(input("Enter number of students: "))
subjects = int(input("Enter number of subjects: "))
if students <= 0 or subjects <= 0:
    print("Invalid dimensions.")
else:
    matrix = []
    names = []
    for i in range(students):
        name = input("Student " + str(i + 1) + " name: ")
        names.append(name)
        row = []
        for j in range(subjects):
            m = float(input("  Subject " + str(j + 1) + " mark
            row.append(m)
        matrix.append(row)
    top_name = ""
    top_total = -1.0
    for i in range(students):
        row_total = 0.0
        for m in matrix[i]:
            row_total += m
        row_avg = row_total / subjects
        print(names[i], "Total:", row_total, "Avg:", round(row_
        if row_total > top_total:
            top_total = row_total
            top_name = names[i]
    print("Top performer:", top_name)
    for j in range(subjects):
        col_total = 0.0
        for i in range(students):
            col_total += matrix[i][j]
        col_avg = col_total / students
        if col_avg < 50:
            print("Low average in Subject", j + 1, ":", round(
```

```
Run  1 ×

D:\Python\.venv\Scripts\python.exe D:\Python\ABL\1.py
Enter number of students: 2
Enter number of subjects: 3
Student 1 name: Arjun
  Subject 1 marks: 80
  Subject 2 marks: 40
  Subject 3 marks: 75
Student 2 name: Priya
  Subject 1 marks: 90
  Subject 2 marks: 50
  Subject 3 marks: 85
Arjun Total: 195.0 Avg: 65.0
Priya Total: 225.0 Avg: 75.0
Top performer: Priya
Low average in Subject 2 : 45.0

Process finished with exit code 0
```



```
students = int(input("Enter number of students: "))
subjects = int(input("Enter number of subjects: "))
if students <= 0 or subjects <= 0:
    print("Invalid dimensions.")
else:
    matrix = []
    names = []
    for i in range(students):
        name = input("Student " + str(i + 1) + " name: ")
        names.append(name)
        row = []
        for j in range(subjects):
            m = float(input("  Subject " + str(j + 1) + " mark
            row.append(m)
        matrix.append(row)
    top_name = ""
    top_total = -1.0
    for i in range(students):
        row_total = 0.0
        for m in matrix[i]:
            row_total += m
        row_avg = row_total / subjects
        print(names[i], "Total:", row_total, "Avg:", round(row_
        if row_total > top_total:
            top_total = row_total
            top_name = names[i]
    print("Top performer:", top_name)
    for j in range(subjects):
        col_total = 0.0
        for i in range(students):
            col_total += matrix[i][j]
        col_avg = col_total / students
        if col_avg < 50:
            print("Low average in Subject", j + 1, ":", round(
```

```
Run  1 ×

D:\Python\.venv\Scripts\python.exe D:\Python\ABL\1.py
Enter number of students: 2
Enter number of subjects: 2
Student 1 name: Kiran
  Subject 1 marks: 88
  Subject 2 marks: 92
Student 2 name: Meera
  Subject 1 marks: 76
  Subject 2 marks: 84
Kiran Total: 180.0 Avg: 90.0
Meera Total: 160.0 Avg: 80.0
Top performer: Kiran

Process finished with exit code 0
```

Conclusion

The program uses nested lists and nested loops to compute row-wise and column-wise matrix statistics, identifying top students and weak subjects.

8.

Tuple-Based Product Catalogue Audit: Store product identifiers, prices, and stock quantities in tuples; detect invalid records, calculate inventory values, and identify items requiring replenishment.

Sol:

Problem statement

Develop a Python program that stores product records as tuples of ID, unit price, and stock quantity, flags invalid entries, calculates total inventory value, and lists products below a minimum stock threshold.

Objectives

- Store product catalogue records as tuples inside a list.
- Unpack product tuples to validate price and stock values.

- Calculate individual and total inventory values for valid products.
- Identify products requiring stock replenishment.

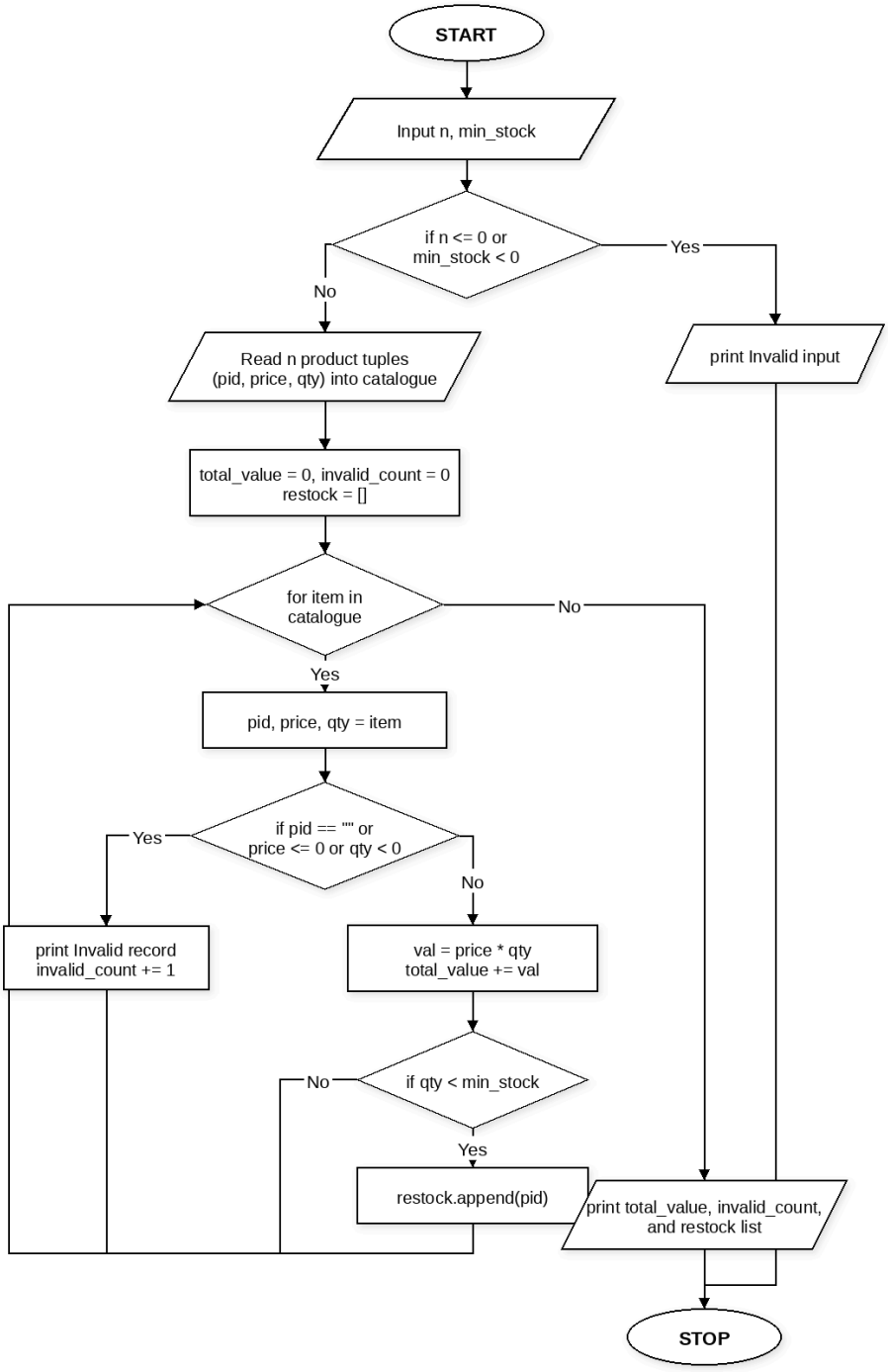
Assumptions

- Each product tuple contains (product_id, unit_price, stock_quantity).
- A record is valid if product_id is non-empty, unit_price is positive, and stock_quantity is non-negative.
- Items with valid stock strictly below the minimum threshold require replenishment.

Algorithm

1. Start.
2. Read the number of products and the minimum stock threshold.
3. Read product ID, unit price, and stock quantity for each product and store as a tuple.
4. Initialize total inventory value and invalid record counter to zero.
5. Unpack each product tuple and validate its fields.
6. If valid, compute item value, add to total inventory value, and check if stock is below threshold.
7. Display item values, total inventory value, invalid count, and replenishment list.
8. Stop.

Flowchart



Pseudocode

START
INPUT n, min_stock
IF n <= 0 OR min_stock < 0
    PRINT "Invalid input."
ELSE

```
    SET catalogue = []
    FOR i from 1 to n
        INPUT pid, price, qty
        APPEND (pid, price, qty) to catalogue
    SET total_value = 0, invalid_count = 0, restock = []
    FOR each item in catalogue
        UNPACK pid, price, qty = item
        IF pid == "" OR price <= 0 OR qty < 0
            PRINT pid, "Invalid record"
            invalid_count = invalid_count + 1
        ELSE
            val = price * qty
            total_value = total_value + val
            PRINT pid, val
            IF qty < min_stock
                APPEND pid to restock
    PRINT total_value, invalid_count, restock
STOP
```

Source Code

```
n = int(input("Enter number of products: "))
min_stock = int(input("Enter minimum stock threshold: "))
if n <= 0 or min_stock < 0:
    print("Invalid input.")
else:
    catalogue = []
    for i in range(1, n + 1):
        pid = input("Product " + str(i) + " ID: ")
        price = float(input("Unit price: "))
        qty = int(input("Stock quantity: "))
        catalogue.append((pid, price, qty))
    total_value = 0.0
    invalid_count = 0
    restock = []
    for item in catalogue:
        pid, price, qty = item
        if pid == "" or price <= 0 or qty < 0:
            print(pid, ": Invalid record")
            invalid_count += 1
        else:
```

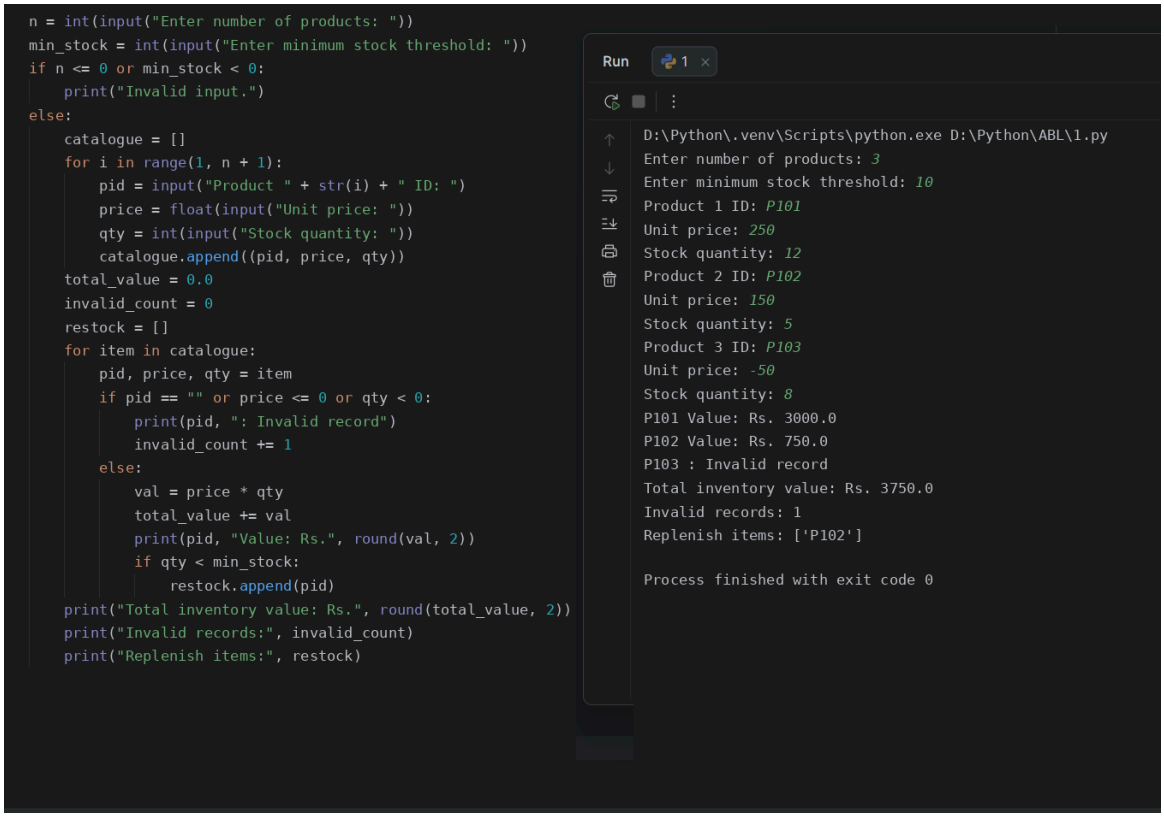
```
        val = price * qty
        total_value += val
        print(pid, "Value: Rs.", round(val, 2))
        if qty < min_stock:
            restock.append(pid)
print("Total inventory value: Rs.", round(total_value, 2))
print("Invalid records:", invalid_count)
print("Replenish items:", restock)
```

Test Cases

1. Enter number of products: 3
   Enter minimum stock threshold: 10
   Product 1 ID: P101
   Unit price: 250
   Stock quantity: 12
   Product 2 ID: P102
   Unit price: 150
   Stock quantity: 5
   Product 3 ID: P103
   Unit price: -50
   Stock quantity: 8
   Output:
   P101 Value: Rs. 3000.0
   P102 Value: Rs. 750.0
   P103 : Invalid record
   Total inventory value: Rs. 3750.0
   Invalid records: 1
   Replenish items: ['P102']
2. Enter number of products: 2
   Enter minimum stock threshold: 15
   Product 1 ID: P201
   Unit price: 500
   Stock quantity: 20
   Product 2 ID: P202
   Unit price: 300
   Stock quantity: 18
   Output:
   P201 Value: Rs. 10000.0
   P202 Value: Rs. 5400.0

Total inventory value: Rs. 15400.0
Invalid records: 0
Replenish items: []

Output

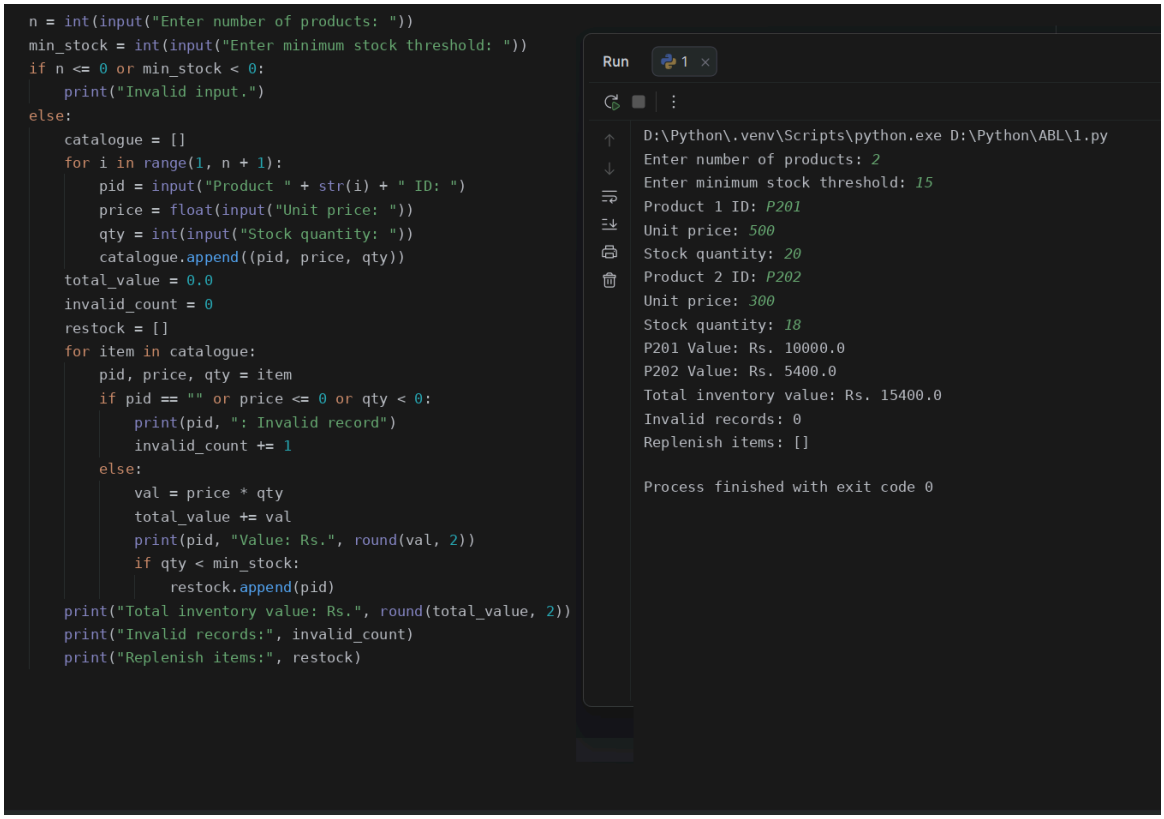


```
n = int(input("Enter number of products: "))
min_stock = int(input("Enter minimum stock threshold: "))
if n <= 0 or min_stock < 0:
    print("Invalid input.")
else:
    catalogue = []
    for i in range(1, n + 1):
        pid = input("Product " + str(i) + " ID: ")
        price = float(input("Unit price: "))
        qty = int(input("Stock quantity: "))
        catalogue.append((pid, price, qty))
    total_value = 0.0
    invalid_count = 0
    restock = []
    for item in catalogue:
        pid, price, qty = item
        if pid == "" or price <= 0 or qty < 0:
            print(pid, ": Invalid record")
            invalid_count += 1
        else:
            val = price * qty
            total_value += val
            print(pid, "Value: Rs.", round(val, 2))
            if qty < min_stock:
                restock.append(pid)
    print("Total inventory value: Rs.", round(total_value, 2))
    print("Invalid records:", invalid_count)
    print("Replenish items:", restock)
```

```
Run  1 ×

D:\Python\.venv\Scripts\python.exe D:\Python\ABL\1.py
Enter number of products: 3
Enter minimum stock threshold: 10
Product 1 ID: P101
Unit price: 250
Stock quantity: 12
Product 2 ID: P102
Unit price: 150
Stock quantity: 5
Product 3 ID: P103
Unit price: -50
Stock quantity: 8
P101 Value: Rs. 3000.0
P102 Value: Rs. 750.0
P103 : Invalid record
Total inventory value: Rs. 3750.0
Invalid records: 1
Replenish items: ['P102']

Process finished with exit code 0
```



Conclusion

The program audits a tuple-based product catalogue using tuple unpacking, validating inventory records and identifying restock requirements.

9.

Student Academic Risk Classification: Combine lists of marks, attendance, and laboratory scores to classify students into risk categories, identify multiple simultaneous risk factors, and generate a summary.

Sol:

Problem statement

Develop a Python program that combines parallel lists of student exam marks, attendance percentages, and laboratory scores to count simultaneous academic risk factors, classify each student into High, Moderate, or Low Risk, and print a summary.

Objectives

- Maintain parallel lists for student names, exam marks, attendance, and lab scores.
- Evaluate multiple simultaneous risk conditions for each student.

- Classify students into High Risk, Moderate Risk, or Low Risk.
- Generate a category-wise summary count.

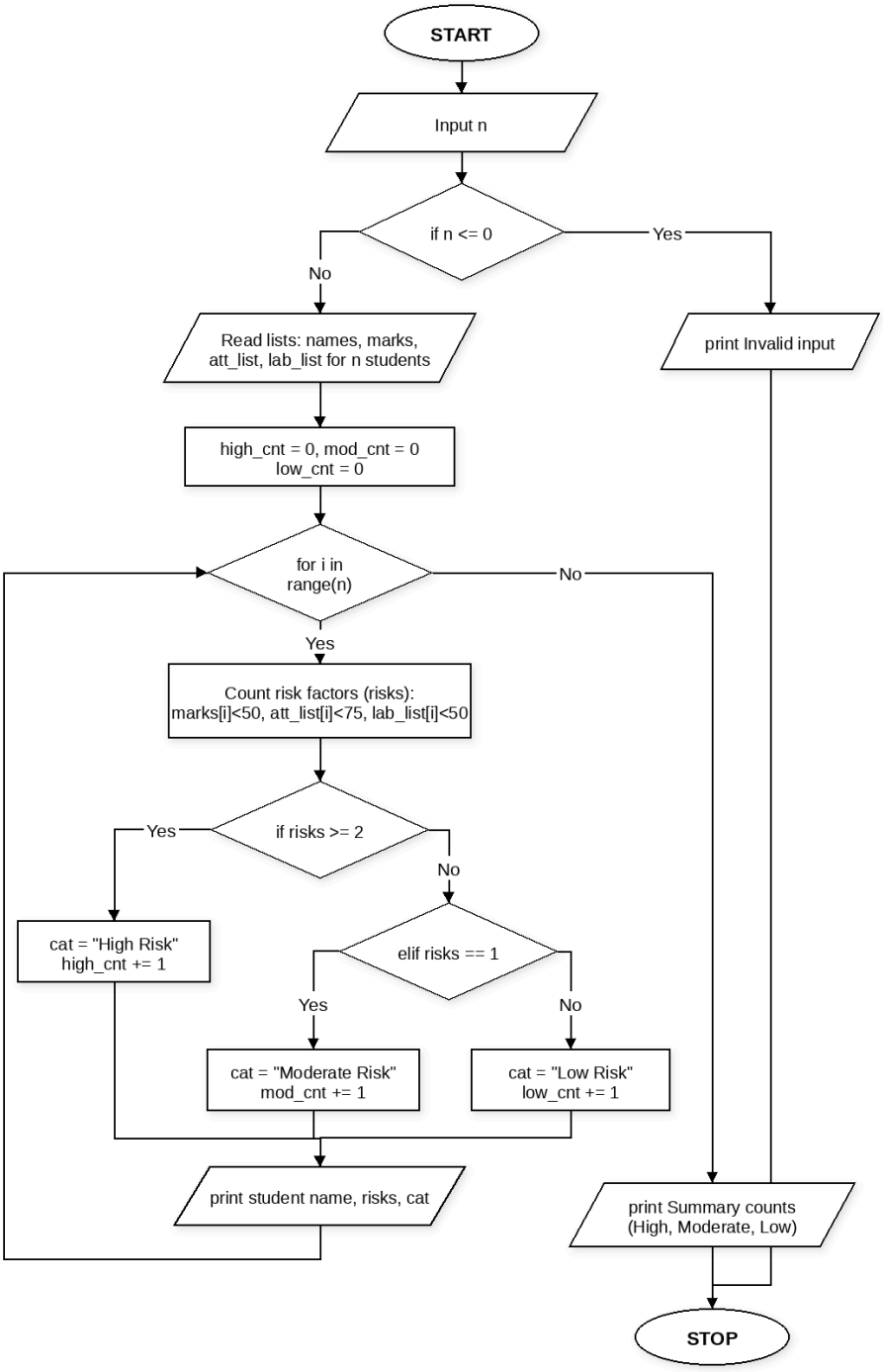
Assumptions

- Risk factors are: exam marks below 50, attendance below 75%, and lab score below 50.
- Two or more risk factors classify a student as High Risk; one factor as Moderate Risk; zero factors as Low Risk.
- All scores and percentages are between 0 and 100.

Algorithm

1. Start.
2. Read the number of students and validate that it is positive.
3. Read each student's name, exam marks, attendance percentage, and lab score into respective lists.
4. Initialize counters for High, Moderate, and Low Risk categories.
5. For each student, count how many risk thresholds are violated.
6. Classify the student based on the number of simultaneous risk factors and update the category count.
7. Display each student's risk factor count and classification, followed by the overall summary.
8. Stop.

Flowchart



Pseudocode

START

INPUT n

IF n <= 0

    PRINT "Invalid input."

ELSE

```
    SET names = [], marks = [], att_list = [], lab_list = []
    FOR i from 1 to n
        INPUT name, mark, att, lab and append to lists
    SET high_cnt = 0, mod_cnt = 0, low_cnt = 0
    FOR i from 0 to n - 1
        SET risks = 0
        IF marks[i] < 50 THEN risks = risks + 1
        IF att_list[i] < 75 THEN risks = risks + 1
        IF lab_list[i] < 50 THEN risks = risks + 1
        IF risks >= 2
            cat = "High Risk", high_cnt = high_cnt + 1
        ELSE IF risks == 1
            cat = "Moderate Risk", mod_cnt = mod_cnt + 1
        ELSE
            cat = "Low Risk", low_cnt = low_cnt + 1
        PRINT names[i], risks, cat
    PRINT high_cnt, mod_cnt, low_cnt
STOP
```

Source Code

```
n = int(input("Enter number of students: "))
if n <= 0:
    print("Invalid input.")
else:
    names = []
    marks = []
    att_list = []
    lab_list = []
    for i in range(1, n + 1):
        names.append(input("Student " + str(i) + " name: "))
        marks.append(float(input(" Exam marks (0-100): ")))
        att_list.append(float(input(" Attendance % (0-100): ")))
        lab_list.append(float(input(" Lab score (0-100): ")))
    high_cnt = 0
    mod_cnt = 0
    low_cnt = 0
    for i in range(n):
        risks = 0
        if marks[i] < 50:
            risks += 1
```

```
    if att_list[i] < 75:
        risks += 1
    if lab_list[i] < 50:
        risks += 1
    if risks >= 2:
        cat = "High Risk"
        high_cnt += 1
    elif risks == 1:
        cat = "Moderate Risk"
        mod_cnt += 1
    else:
        cat = "Low Risk"
        low_cnt += 1
    print(names[i], "- Factors:", risks, "-", cat)
print("Summary -> High:", high_cnt, "Moderate:", mod_cnt, "Low:", low_cnt)
```

Test Cases

1. Enter number of students: 3
   Student 1 name: Ravi (Marks: 42, Attendance: 68, Lab: 55)
   Student 2 name: Sneha (Marks: 72, Attendance: 70, Lab: 80)
   Student 3 name: Vikram (Marks: 85, Attendance: 90, Lab: 88)
   Output:
   Ravi - Factors: 2 - High Risk
   Sneha - Factors: 1 - Moderate Risk
   Vikram - Factors: 0 - Low Risk
   Summary -> High: 1 Moderate: 1 Low: 1
2. Enter number of students: 2
   Student 1 name: Ananya (Marks: 35, Attendance: 60, Lab: 40)
   Student 2 name: Dev (Marks: 78, Attendance: 82, Lab: 75)
   Output:
   Ananya - Factors: 3 - High Risk
   Dev - Factors: 0 - Low Risk
   Summary -> High: 1 Moderate: 0 Low: 1

Output

```python
n = int(input("Enter number of students: "))
if n <= 0:
    print("Invalid input.")
else:
    names = []
    marks = []
    att_list = []
    lab_list = []
    for i in range(1, n + 1):
        names.append(input("Student " + str(i) + " name: "))
        marks.append(float(input(" Exam marks (0-100): ")))
        att_list.append(float(input(" Attendance % (0-100): ")))
        lab_list.append(float(input(" Lab score (0-100): ")))
    high_cnt = 0
    mod_cnt = 0
    low_cnt = 0
    for i in range(n):
        risks = 0
        if marks[i] < 50:
            risks += 1
        if att_list[i] < 75:
            risks += 1
        if lab_list[i] < 50:
            risks += 1
        if risks >= 2:
            cat = "High Risk"
            high_cnt += 1
        elif risks == 1:
            cat = "Moderate Risk"
            mod_cnt += 1
        else:
            cat = "Low Risk"
            low_cnt += 1
        print(names[i], "- Factors:", risks, "-", cat)
    print("Summary -> High:", high_cnt, "Moderate:", mod_cnt, '
```

```
Run  1
D:\Python\.venv\Scripts\python.exe D:\Python\ABL\1.py
Enter number of students: 3
Student 1 name: Ravi
 Exam marks (0-100): 42
 Attendance % (0-100): 68
 Lab score (0-100): 55
Student 2 name: Sneha
 Exam marks (0-100): 72
 Attendance % (0-100): 70
 Lab score (0-100): 80
Student 3 name: Vikram
 Exam marks (0-100): 85
 Attendance % (0-100): 90
 Lab score (0-100): 88
Ravi - Factors: 2 - High Risk
Sneha - Factors: 1 - Moderate Risk
Vikram - Factors: 0 - Low Risk
Summary -> High: 1 Moderate: 1 Low: 1

Process finished with exit code 0
```

```python
n = int(input("Enter number of students: "))
if n <= 0:
    print("Invalid input.")
else:
    names = []
    marks = []
    att_list = []
    lab_list = []
    for i in range(1, n + 1):
        names.append(input("Student " + str(i) + " name: "))
        marks.append(float(input(" Exam marks (0-100): ")))
        att_list.append(float(input(" Attendance % (0-100): ")))
        lab_list.append(float(input(" Lab score (0-100): ")))
    high_cnt = 0
    mod_cnt = 0
    low_cnt = 0
    for i in range(n):
        risks = 0
        if marks[i] < 50:
            risks += 1
        if att_list[i] < 75:
            risks += 1
        if lab_list[i] < 50:
            risks += 1
        if risks >= 2:
            cat = "High Risk"
            high_cnt += 1
        elif risks == 1:
            cat = "Moderate Risk"
            mod_cnt += 1
        else:
            cat = "Low Risk"
            low_cnt += 1
        print(names[i], "- Factors:", risks, "-", cat)
    print("Summary -> High:", high_cnt, "Moderate:", mod_cnt, '
```

```
Run  1
D:\Python\.venv\Scripts\python.exe D:\Python\ABL\1.py
Enter number of students: 2
Student 1 name: Ananya
 Exam marks (0-100): 35
 Attendance % (0-100): 60
 Lab score (0-100): 40
Student 2 name: Dev
 Exam marks (0-100): 78
 Attendance % (0-100): 82
 Lab score (0-100): 75
Ananya - Factors: 3 - High Risk
Dev - Factors: 0 - Low Risk
Summary -> High: 1 Moderate: 0 Low: 1

Process finished with exit code 0
```

Conclusion

The program combines multiple academic indicators stored in lists to detect simultaneous risk factors and classify students into risk tiers.

10.

Smart Classroom Timetable Allocation: Represent courses, rooms, and periods using lists and tuples; validate classroom capacities, detect clashes, and check whether all courses can be accommodated under the stated constraints.

Sol:

Problem statement

Develop a Python program that stores classroom capacities and course requests using lists and tuples, assigns each course to the first available room that meets capacity without period clashes, and reports whether all courses can be accommodated.

Objectives

- Represent classrooms and capacities as tuples in a list.
- Validate room capacity and check for period clashes before allocation.
- Track allocated course-room-period tuples and unaccommodated courses.
- Determine whether all courses can be accommodated under the given constraints.

Assumptions

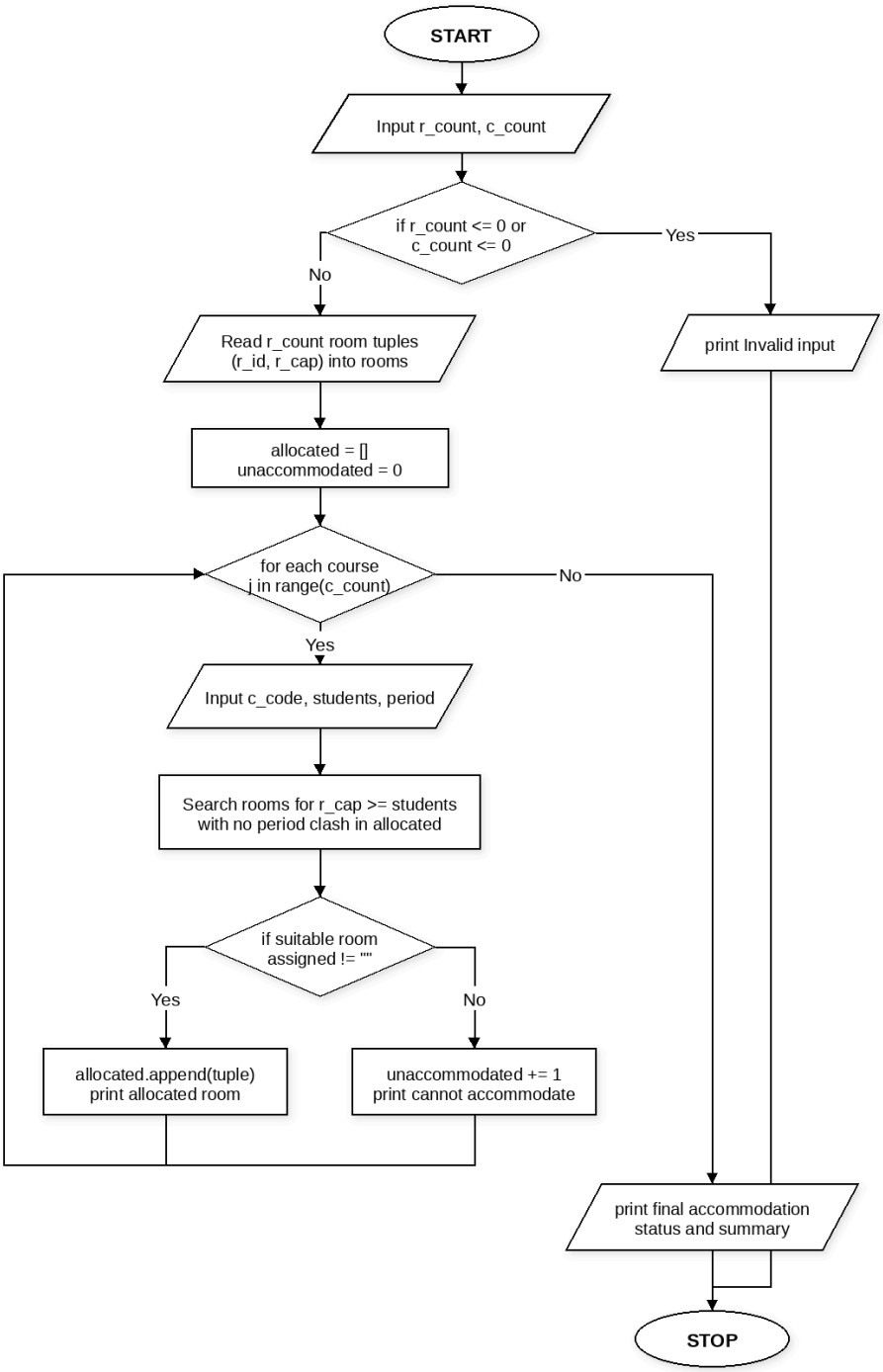
- Each room tuple stores (room_name, capacity).
- A room can host at most one course in any given period (1 to 6).
- A course is assigned to the first room in list order that has sufficient capacity and is free in the requested period.

Algorithm

1. Start.
2. Input the number of rooms and number of courses.
3. Read each room's identifier and seating capacity and store as a tuple in the rooms list.
4. Initialize an empty list for allocated slots and set unaccommodated count to zero.
5. For each course, read the course code, enrolled students, and requested period.

6. Iterate through the rooms list using tuple unpacking to find a room with capacity >= enrolled students that is not already booked in that period.
7. If a valid room is found, record the allocation tuple; otherwise, increment the unaccommodated count.
8. Display the allocation result for each course and the final accommodation status.
9. Stop.

Flowchart



Pseudocode

```
START
INPUT r_count, c_count
IF r_count <= 0 OR c_count <= 0
   PRINT "Invalid input."
ELSE
   SET rooms = []
   FOR i from 1 to r_count
      INPUT r_id, r_cap and append (r_id, r_cap) to rooms
   SET allocated = [], unaccommodated = 0
   FOR j from 1 to c_count
      INPUT c_code, students, period
      SET assigned = ""
      FOR each (r_id, r_cap) in rooms
         IF r_cap >= students AND assigned == ""
            CHECK if (r_id, period) is already in allocated
            IF not booked THEN assigned = r_id
      IF assigned != ""
         APPEND (c_code, assigned, period) to allocated
         PRINT c_code, "allocated to", assigned
      ELSE
         unaccommodated = unaccommodated + 1
         PRINT c_code, "cannot be accommodated"
   IF unaccommodated == 0
      PRINT "Result: All courses accommodated"
   ELSE
      PRINT "Result: Unaccommodated courses:", unaccommodated
STOP
```

Source Code

```
r_count = int(input("Enter number of rooms: "))
c_count = int(input("Enter number of courses: "))
if r_count <= 0 or c_count <= 0:
   print("Invalid input.")
else:
   rooms = []
   for i in range(1, r_count + 1):
      r_id = input("Room " + str(i) + " name: ").upper()
      r_cap = int(input("Room " + str(i) + " capacity: "))
```

```
    rooms.append((r_id, r_cap))
allocated = []
unaccommodated = 0
for j in range(1, c_count + 1):
    c_code = input("Course " + str(j) + " code: ").upper()
    students = int(input("Enrolled students: "))
    period = int(input("Requested period (1-6): "))
    assigned = ""
    for rm in rooms:
        r_id, r_cap = rm
        if r_cap >= students and assigned == "":
            clash = False
            for slot in allocated:
                al_c, al_r, al_p = slot
                if al_r == r_id and al_p == period:
                    clash = True
            if not clash:
                assigned = r_id
    if assigned != "":
        allocated.append((c_code, assigned, period))
        print(c_code, "allocated to", assigned)
    else:
        unaccommodated += 1
        print(c_code, "cannot be accommodated")
if unaccommodated == 0:
    print("Result: All courses accommodated")
else:
    print("Unaccommodated:", unaccommodated)
```

Test Cases

1. Enter number of rooms: 2 (R101: 40, R102: 60)
   Enter number of courses: 3
   Course 1: CS101, Students: 35, Period: 1
   Course 2: CS102, Students: 50, Period: 1
   Course 3: CS103, Students: 45, Period: 1
   Output:
   CS101 allocated to R101
   CS102 allocated to R102
   CS103 cannot be accommodated
   Unaccommodated: 1

2. Enter number of rooms: 2 (R201: 50, R202: 80)
   Enter number of courses: 2
   Course 1: AI101, Students: 45, Period: 2
   Course 2: DS102, Students: 70, Period: 2
   Output:
   AI101 allocated to R201
   DS102 allocated to R202
   Result: All courses accommodated

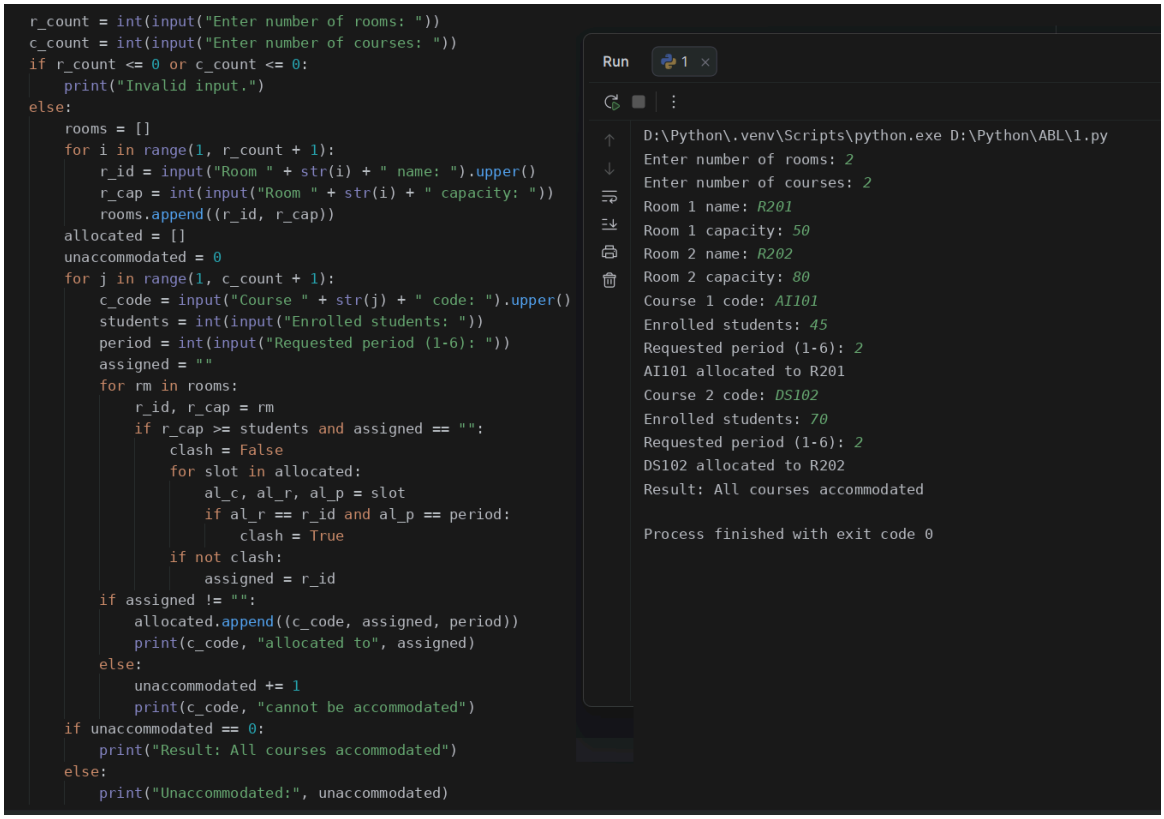
Output

```
r_count = int(input("Enter number of rooms: "))
c_count = int(input("Enter number of courses: "))
if r_count <= 0 or c_count <= 0:
    print("Invalid input.")
else:
    rooms = []
    for i in range(1, r_count + 1):
        r_id = input("Room " + str(i) + " name: ").upper()
        r_cap = int(input("Room " + str(i) + " capacity: "))
        rooms.append((r_id, r_cap))
    allocated = []
    unaccommodated = 0
    for j in range(1, c_count + 1):
        c_code = input("Course " + str(j) + " code: ").upper()
        students = int(input("Enrolled students: "))
        period = int(input("Requested period (1-6): "))
        assigned = ""
        for rm in rooms:
            r_id, r_cap = rm
            if r_cap >= students and assigned == "":
                clash = False
                for slot in allocated:
                    al_c, al_r, al_p = slot
                    if al_r == r_id and al_p == period:
                        clash = True
                if not clash:
                    assigned = r_id
        if assigned != "":
            allocated.append((c_code, assigned, period))
            print(c_code, "allocated to", assigned)
        else:
            unaccommodated += 1
            print(c_code, "cannot be accommodated")
    if unaccommodated == 0:
        print("Result: All courses accommodated")
    else:
        print("Unaccommodated:", unaccommodated)
```

```
Run  1 ×

D:\Python\.venv\Scripts\python.exe D:\Python\ABL\1.py
Enter number of rooms: 2
Enter number of courses: 3
Room 1 name: R101
Room 1 capacity: 40
Room 2 name: R102
Room 2 capacity: 60
Course 1 code: CS101
Enrolled students: 35
Requested period (1-6): 1
CS101 allocated to R101
Course 2 code: CS102
Enrolled students: 50
Requested period (1-6): 1
CS102 allocated to R102
Course 3 code: CS103
Enrolled students: 45
Requested period (1-6): 1
CS103 cannot be accommodated
Unaccommodated: 1

Process finished with exit code 0
```



```python
r_count = int(input("Enter number of rooms: "))
c_count = int(input("Enter number of courses: "))
if r_count <= 0 or c_count <= 0:
    print("Invalid input.")
else:
    rooms = []
    for i in range(1, r_count + 1):
        r_id = input("Room " + str(i) + " name: ").upper()
        r_cap = int(input("Room " + str(i) + " capacity: "))
        rooms.append((r_id, r_cap))
    allocated = []
    unaccommodated = 0
    for j in range(1, c_count + 1):
        c_code = input("Course " + str(j) + " code: ").upper()
        students = int(input("Enrolled students: "))
        period = int(input("Requested period (1-6): "))
        assigned = ""
        for rm in rooms:
            r_id, r_cap = rm
            if r_cap >= students and assigned == "":
                clash = False
                for slot in allocated:
                    al_c, al_r, al_p = slot
                    if al_r == r_id and al_p == period:
                        clash = True
                if not clash:
                    assigned = r_id
        if assigned != "":
            allocated.append((c_code, assigned, period))
            print(c_code, "allocated to", assigned)
        else:
            unaccommodated += 1
            print(c_code, "cannot be accommodated")
    if unaccommodated == 0:
        print("Result: All courses accommodated")
    else:
        print("Unaccommodated:", unaccommodated)
```

```
Run  1

D:\Python\.venv\Scripts\python.exe D:\Python\ABL\1.py
Enter number of rooms: 2
Enter number of courses: 2
Room 1 name: R201
Room 1 capacity: 50
Room 2 name: R202
Room 2 capacity: 80
Course 1 code: AI101
Enrolled students: 45
Requested period (1-6): 2
AI101 allocated to R201
Course 2 code: DS102
Enrolled students: 70
Requested period (1-6): 2
DS102 allocated to R202
Result: All courses accommodated

Process finished with exit code 0
```

Conclusion

The program allocates classrooms using lists, tuples, and tuple unpacking while enforcing room capacity and period clash constraints.